AI-Powered Career Growth and Productivity Platform

Major project report submitted in partial full fillment of the requirement
for the degree of
Bachelor of Technology
In
Computer Science and Engineering

By

Priyansh Lunawat (221030169)

Udit Sharma (221030199)

Ashish Agarwal (221030420)

UNDER THE SUPERVISION OF

Mr. Aayush Sharma (Assistant Professor (Grade-I))

Department of Computer Science & Engineering and Information Technology

Jaypee University Of Information Technology, Waknaghat, 173234, Himachal Pradesh, (INDIA)

May 2026

SUPERVISOR'S CERTIFICATE

This is to certify that the major project report entitled **'Project Title'**, submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering**, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is a bonafide project work carried out under my supervision during the period from July 2025 to May 2026.

I have personally supervised the research work and confirm that it meets the standards required for submission. The project work has been conducted in accordance with ethical guidelines, and the matter embodied in the report has not been submitted elsewhere for the award of any other degree or diploma

Supervisor Name & Sign: Mr. Aayush Sharma

Date: 08/05/2026 **Designation:** Assistant Professor (Grade-I)

Place: Waknaghat, Solan **Department:** Computer Science & Engineering

CANDIDATE'S DECLARATION

We hereby declare that the work presented in this major project report entitled **'AI-Powered Career Growth and Productivity Platform'**, submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering**, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is an authentic record of our own work carried out during the period from July 2025 to July 2026 under the supervision of **Mr. Aayush Sharma**.

We further declare that the matter embodied in this report has not been submitted for the award of any other degree or diploma at any other university or institution.

Name & Sign: Priyansh Lunawat
Roll No.: 221030169
Date: 08/05/2026

Name & Sign: Udit Sharma
Roll No.: 221030199
Date: 08/05/2026

Name & Sign: Ashish Agarwal
Roll No.: 221030420
Date: 08/05/2026

This is to certify that the above statement made by the candidates is true to the best of my knowledge.

Supervisor Name & Sign: Mr. Aayush Sharma
Designation: Assistant Professor (Grade-I)
Department: Computer Science & Engineering

Date: 08/05/2026
Place: Waknaghat,Solan

ACKNOWLEDGEMENT

Firstly, We express our heartiest thanks and gratefulness to almighty God for His divine blessing makes us possible to complete the project work successfully.

We are really grateful and wish our profound indebtedness to Supervisor **Mr. Aayush Sharma**, Department of CSE Jaypee University of Information Technology, Wakhnaghat. Deep Knowledge & keen interest of my supervisor in the field of "**Artificial Intelligence**" to carry out this project. His endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts and correcting them at all stages have made it possible to complete this project.

We would also generously welcome each one of those individuals who have helped us straight forwardly or in a roundabout way in making this project a win. In this unique situation, we might want to thank the various staff individuals, both educating and non-instructing, which have developed their convenient help and facilitated our undertaking.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

Priyansh Lunawat (221030169)

Udit Sharma (221030199)

Ashish Agarwal (22103420)

TABLE OF CONTENT

S.No.	Title ..... PageNo.
1	SUPERRVISOR'S CERTIFICATE i
2	CANDIDATE'S DECLARATION ii
3	ACKNOWLEDGEMENT iii
4	LIST OF FIGURES vi
5	LIST OF TABLES viii
6	ABSTRACT ix
7	CHAPTER 1: INTRODUCTION 1
8	1.1 Introduction 1
9	1.2 Problem Statement 3
10	1.3 Objectives 4
11	1.4 Significance and Motivation of the Project Work 4
12	1.5 Organization of Project Report 5
13	CHAPTER 2: LITERATURE 6
14	2.1 Overview of Relevant Literature 6
15	2.2 Related Work 7
16	2.3 Key Gaps in the Literature 10
17	CHAPTER 3 : SYSTEM DEVELOPMENT 11
18	3.1 Requirements and Analysis 11
19	3.2 Project Design and Architecture 12
20	3.3 Tools and Technologies 15
21	3.4 Data Preparation 15
22	3.5 Ollama & Llama 3.x Model Architecture 17
23	3.6 Implementation 18
24	3.7 Key Challenges 39
25	CHAPTER 4 : TESTING 40
26	4.1 Testing Strategy 40

27 4.2 Test Cases and Outcomes.....41

28 CHAPTER 5: RESULTS AND EVALUATION 43

29 5.1 Results.....43

30 5.2 Comparison with Existing Solution 54

31 CHAPTER 6: CONCLUSIONS AND FUTURE SCOPE 56

32 6.1 Conclusion 56

33 6.2 Future Scope.....57

34 List Of Publications 58

35 REFRENCES 59

36 APPENDIX..... 62

LIST OF FIGURES

Fig. No.	Figure Title	Page No.
3.1	System Architecture of Career Companion	14
3.2	Llama 3.6 Model Architecture	18
3.3	Website Frontend	20
3.4	Flowchart of Resume Generator with Job Prediction	21
3.5	Flowchart of Cover Letter Generator	22
3.6	Flowchart of Job Application Tracker	23
3.7	AI -Based Mock Interview System Workflow	24
3.8	Question Form Component in Frontend	25
3.9	Backend Environment Variable and Router Configuration	26
3.10	Frontend Entry File and API Configuration	27
3.11	preprocess_cv.py (DOCX to PDF Conversion)	28
3.12	regex_parser.py (Contact Info & Chunk Extraction)	29
3.13	llm_parser.py (LLM-Based Resume Processing)	30
3.14	config.py (System Configuration & Directory Setup)	31
3.15	Resume Analysis Module Using LLM	32
3.16	Main Parsing Pipeline Initialization	33
3.17	RAG-Powered Section Extraction	34

3.18	clean_text Preprocessing Function	35
3.19	Handle Interview Chat Function	36
3.20	Interview Setup Function	37
3.21	Interview Chat Function	38
5.1	structured resume	44
5.2	Job application Tracker	45
5.3	MongoDB Database Records	47
5.4	PDF Text-Extraction Console Output	48
5.5	Pdf Text cleaning	49
5.6	AI Interview Interface	50
5.7	AI Interview Feedback Report	52
5.8	AI Interview Feedback Recommendations	52
1	Resume Analysis Function (Backend)	62
2	React API Configuration	63

LIST OF TABLES

T. No.	Table Title	Page No.
2.1	Key Research Works	7
4.1	Test Case Table	41
5.1	Performance Comparison of Resume Parsing Models	53

ABSTRACT

The project **AI-Powered Career Growth and Productivity Platform** is a conceptualized initiative aimed at enhancing the career development of students, job seekers, and working professionals through a unified and intelligent system. The shift to technology-oriented recruiting has served to change the requirements of a candidate. Contrary to previous recruitment where meetings were conducted in person, paper and pen evaluation and flexible evaluation methodologies, contemporary hiring is more organized. Companies typically get hundreds of applications for each role and to manage this influx they utilize automated tools that screen, sort and score candidates prior, to any recruiter reviewing their dossiers. Consequently the way information is presented. In résumés cover letters and answers during interviews. Now has a significantly greater impact, on deciding if a candidate advances to the subsequent phase.

CHAPTER 1: INTRODUCTION

1.1 Introduction:

The transition to tech-focused recruitment has radically altered the candidate profile [1][2][4]. Unlike traditional recruitment that was in-person interactions, handwritten assessment and diverse evaluation techniques, present-day hiring is more structured [24][30]. Organizations usually receive multiple applications for each position and to handle this overwhelming number they employ automatic tools that filter, arrange and grade candidates well before a recruiter actually going through their folders. Therefore the manner of displaying data. In CVs motivation letters and responses in interviews. At present dramatically influences the selection of a candidate for the next round.

Yet, quite a number of students and fresh graduates experience difficulty in meeting these requirements for various reasons [13][19]. One of the main issues is knowing how applicant tracking systems (ATS) score resumes [21][24]. Many job seekers unintentionally make documents that look good to the human eye but are not well suited to the ATS standards. In addition to this, one must be able to present their talent and skills succinctly and effectively. It is a real challenge to compose a rsum or cover letter that is both honest and compelling without the help of a professional. Similarly, preparing for interviews entails practicing, being confident and knowing what kinds of questions most employers are likely to ask. Most of the time, new students do not have interview sessions as a part of their curriculum in a reallife setting [1][13].

This initiative offers a comprehensive AI-based platform that supports job seekers throughout their entire journey, especially in key moments, without merely flooding them with numerous tools or instructions. Instead, it guides them through each step. From making a rsum to finishing up the story and writing cover letters tailored for specific jobs to even having practice sessions for interviews, all these could be done within a single interface [1][3]. In fact, the system's main goal is not to override or change human decisions but rather to serve as a dependable, always available assistant whom the students would really trust and feel at ease when the students are working independently.

The main idea behind developing the platform was that it can be highly modular. So various modules are working on different tasks, e. g. preparing a resume, reviewing content, conducting a mock interview, and tracking a job. Modules are talking to each other through well-defined APIs that already give the system the capability of being expanded later on. Besides providing the platform with more options to be flexible, using AI and NLP for understanding and generating the text simultaneously increases its ability to understand and adapt to different kinds of user needs.

This project is here for job seekers in order to alleviate their stress and confusion when finding a job [5][19]. Through the portal, they will get straightforward and easy-to-understand advice that will be customized and also aligned with the latest recruitment norms. In short, this project is about making one's transition from student to the working world smoother since it gives people the tools to confidently compete in the ever-changing professional world.

Moreover, the platform highlights the importance of learning and receiving feedback as ongoing activities, which is very essential in a highly competitive job market. It never thought of making a resume or practicing for an interview as one time tasks only. In fact, it still encourages users to make continual changes and improvements. Individuals may return their profile pages, include new achievements, and modify their responses relying on AI suggestions and analysis of their performance data. Such a recursive feedback loop helps the users to discover their strengths and weaknesses which gradually, results in the development of their job-related skills. Additionally, by tracking the users' changes, the platform boosts their confidence as well as helps them to better understand their career path[5][13].

On the other hand, the platform extends career guidance services to a wider audience regardless of their background or situation through tech-enabled professional advisory services. Students particularly those from underrepresented or hard-to-reach areas are the ones that not only miss the chance to meet career counselors and mentors but also generally do not have access to graduate-level training, career counseling, or job placement services. This product with its well-structured and easy-to-follow instructions can support the wide range of users, thus solving the problem.. Users lacking the tech understanding totally are also theoretically the target audience

as the design, being extremely user-friendly, outlines the pathways to navigating the interface and making the most out of the server functionalities which, to summarize, is making high-quality career training tools universally accessible

In the future, the system may increase its capabilities by including a few unique features such as carrying out live labor market analyses, giving users customized career advice, and even integrating with professional networking sites [5][19]. The system, as the ways of recruiting change, will be able to keep up by refreshing its models and being aware of the latest hiring patterns, thereby making sure that the users are always in line with the market. In the end, this project is not only addressing the need for job applications at the moment but also, through guiding long- term career planning, enabling users to make well-thought-out decisions and continue growing in a professional environment that is constantly changing [19].

1.2 Problem Statement

Students will often struggle with putting together career documentation because of not being advised or getting the right knowledge of what the field requires. Many students are confused, as to what a recruiter wants, how to present the best of themselves, or how to compose a professional document. Most of the resources include merely basic templates, which in most cases are not in line with the unique abilities, work experience and career expectations of specific students. The other problem is that no single platform unites all the resources that are important in career development. The students should be enabled to use different sites to draft resumes, draft cover letters to prepare to attend the job interview and get job-related tips. Such a disjointed approach does not waste time, but generates discrepancies, in the products they make.

Due to all these constraints, it is apparent that a single, user-friendly web system is necessary that will assist students in their career preparation process. An AI-driven platform can greatly reduce the workload and improve the quality by customizing the resume, matching them with appropriate job opportunities, generating professional cover letters as well as helping them prepare for interviews. In addition, a system like this can be used to assist students step by step and also make the whole process more efficient by simplifying it.

1.3 Objectives:

The main objectives of the project are:

- To develop a user-friendly web application that helps students create professional career documents with ease.
- To generate personalized resumes and cover letters using AI-based suggestions tailored to a student's skills and background.
- To provide an AI-powered job-role recommendation feature that guides students toward suitable career options.
- To build a Job Application Tracker that allows students to manage and monitor all their internship and job applications in one place.
- To offer all these features within a single integrated platform, making the placement preparation process more organized, efficient, and accessible.
- To ensure that the system is easy to use, even for students with little technical experience

1.4 Significance and Motivation of the Project Work:

Students often spend many hours creating career documents using pen and paper. Lack of support the task becomes a slow, stressful and sometimes demoralizing experience. A lot of students do not have access to resume advisors, mentors or industry specialists that could help them polish their documents. As a result very talented students have a hard time demonstrating their achievements well.

The inspiration developed for this project was witnessing these issues faced by peers and juniors, throughout internship season. The lack of tailored instruments caused students to rely on templates which look similar and fail to convey distinctive strengths.

Through AI integration the platform not only helps in creating content but also helps students to explore career paths that they may have not been able to explore before. This is added value, given that educational institutions with hundreds of students seeking placements every year. Additionally the system allows for regularity, structure in formatting and accuracy ensuring that students will present documents that reflect them well.

This project helps in bridging the gap between the preparation of students and the expected

in industry. It brings together the technology, usability and academic needs in a way that supports students and increases their chances of success.

1.5 Organization of Project Report:

The whole project report is divided into six chapters. Each one covers a separate phase of the system's development:

- **Chapter 1: Introduction:**
  Presents background information, justification for the project, the problem and objectives. the objectives and motivation

- **Chapter 2: Literature Survey:**
  Reviews existing research, tools, and recent developments connected to resume generation, cover letter automation, AI-based recommendation systems and job applications platforms

- **Chapter 3: System Development:**
  Covers system requirements, architecture, backend and frontend design, algorithms, data flow and details on implementation.

- **Chapter 4: Testing:**
  Describes testing strategy, test cases, applied testing methods and results obtained during system test.

- **Chapter 5: Results and Evaluation:**
  Conclude by discussing system performance, observed behavior and give a comparison with extant solutions.

- **Chapter 6: Conclusion and Future Scope:**
  Concludes the project, sums up all things done and outlines the future work plan. improvements and extension of features

CHAPTER 2: LITERATURE

This chapter focus on research, tools and techniques in AI-guided career counseling resume, interview and coding test optimization.Based on a review of previous studies, I pinpoint the weaknesses that my system intends to tackle.

2.1 Overview of Relevant Literature:

Recently, resume parsing, AI-generated content, and career help tools on the web are the focus of intensive research.

Most of the latest studies and tech forum articles indicate that students and job seekers increasingly depend on automatic systems for resume making, cover letter creating, and job matching. The primary factors behind this trend have been the enhancements in NLP models and access to big, pre-trained language models.

- **Resume parsing and extraction:** Earlier systems mostly depended on fixed templates or simple rules to read resumes. These worked only when the resume format was predictable. Recent research (especially during the last 4–5 years) shows a major shift towards using deep-learning models like BERT to understand resume text more accurately. These models perform better when resumes have different layouts, fonts, or PDF structures. Many recent publications also combine OCR with NLP to handle scanned resumes [9][21].
- **Resume and cover-letter generation:** The traditional method was mostly template-based: users filled in a form and the system generated a standard resume or cover letter. There is recent work discussing the use of transformer based models to generate more natural and personalised content [23][29]. These systems have the ability to analyze the user's skills and experiences and rewriting it in a more professional manner. There are also several studies that layout that AI-generated content result in not only an increase in readability but also minimizing errors and saving time for students.
- **Career Role Recommendation:** Modern job recommendation research is focused on skills-job description similarity using embedding-based similarity. Instead of the keyword matching, recent work is done using vector representations of skills, responsibilities, and experience [16][22]. Many studies combine this with occupational databases or knowledge graphs in order to better the accuracy of suggested roles.

- **Interview preparation tools:** Although this field is still growing, recent work suggests the use of conversational agents and feedback systems to help students practice interview questions. These tools usually apply NLP to analyse answers and offer some basic suggestions like clarity, structure and relevance. Research papers also mention the difficulties in giving human-like feedback which is still an open problem.
- **Industry practices and cloud-based systems:** White papers and technical blogs from large cloud providers explain how real platforms are built in the real world: using microservices, NoSQL databases, as well as separate frontend - backend layers; These resources also focus on the security common practices, user authentication and scalability which are followed in modern web systems [24][30].
- **Identified gap:** The literature indicates that there is a gap between student needs and the availability of tools that will help students learn. use different platforms for different things - one for building a resume, one for career suggestions of and another for practising interview. There is hardly any unified system that brings all these features in a simple and student-friendly manner. This disparity in current solutions supports the concept of our project.

2.2 Related Work:

Table 2.1 : Key Research Works

S.No.	Author & Paper Title	Journal/ Conference (Year)	Tools/ Techniques/ Dataset	Key Findings/ Results
1	Nguyen, T. T. H. et al. – SimInterview: Transforming Business Education through LLM-Based Simulated Multilingual Interview Training System	arXiv (2025)	LLMs, Whisper STT, GPT-SoVITS, Ditto avatar, ChromaDB, RAG	Multilingual simulated interviews; high user satisfaction; better interview readiness
2	Koshti, H. et al. – *AI*-Powered Interview Preparation System: Integrating Resume Analysis, HR Simulation, and Technical Skill Assessment	JERR (2025)	CNN for emotion detection, NLP for resume analysis, Speech Recognition, ATS integration	Improved candidate preparation efficiency; objective evaluation; real-time feedback

3	Gayathri Devi M. et al. – AI Friendly Resume	IJSREM (2024)	AI/NLP for keyword extraction, semantic matching, formatting	Increased resume shortlisting rates compared to traditional formats
4	AI-Based Mock Interview Platform	IRJMETS (2025)	Smart resume parsing, emotion detection, dynamic question generation	Personalized interview practice; real-time feedback;
5	Sharma & Jain – Code Evaluation and Suggestion System	IERJ (2025)	Automated code evaluation, suggestion modules	Evaluates submitted code and provides improvement suggestions
6	Resume Generator Using AI	IRJMETS (2025)	Flask, NLP, ML, template systems	AI-assisted resume structuring, optimization, and output
7	Automated Code Assessment for Education: Review and Perspectives	MDPI (2022)	Static analysis, test-based, ML-based feedback	Strengths, limitations, and trends in automated code assessment systems
8	Speeding Up Automated Assessment of Programming Exercises	ACM (2022)	Caching, ML prediction, static analysis	Reduced latency in grading; improved scalability
9	Sanyal, K. et al. – Intelligent Resume Parsing and Job Recommendation via Web-Based CV Analysis System	IJRASET (2025)	NLP (pdf parsing, regex), Flask, job search API	Extracts structured info from resumes; real-time job recommendations
10	Korrapati, L. N. V. Babu et al. – A Machine Learning Approach for Automation of Resume Recommendation System	IJRASET (2022)	Text mining, resume classification	Classifies resumes, automates recommendation; speeds up candidate screening

11	Varshith Reddy, K. S. et al. – Resume Analyzer and Job Recommendation System	IRE Journals (2025)	OCR, NLP, TF-IDF, Cosine Similarity, KNN	Structured resume data; ranks candidates; suggests learning paths
12	Jiang, J. et al. – Learning Effective Representations for Person-Job Fit by Feature Fusion	arXiv (2020)	Deep learning, feature fusion, LSTM	Improved candidate-job matching accuracy using explicit and
				implicit features
13	Rahman, M. et al. – Artificial Intelligence in Career Counseling: A Test Case with ResumAI	arXiv (2023)	AI chatbot, resume feedback	ResumAI effective in improving resumes; supports AI in career counseling
14	Apaza, H. et al. – Job Recommendation Based on Curriculum Vitae Using Text Mining	Springer FICC (2021)	Text mining, similarity measures	Matches CVs with job descriptions; reduces manual screening workload
15	Messer, M. et al. – Automated Grading and Feedback Tools for Programming Education: A Systematic Review	arXiv (2023)	Systematic review of automated tools	Categorizes grading tools; identifies strengths and limitations
16	Li, C. et al. – Competence-Level Prediction and Resume & Job Description Matching Using Transformer Models	arXiv (2020)	Transformer models, section encoding	Accurate competence prediction and job matching using context-aware models
17	Patil, R. et al. – Automated Assessment of Multimodal Answer Sheets in the STEM Domain	Papers With Code (2024)	LLMs, YoloV5, OCR	Evaluates text + diagrams; improves assessment of complex answers
18	Khan, M. S. et al. – Automated Assessment for C/C++ Programming in ODL Environment	Papers With Code (2022)	Auto grading, feedback for C/C++	Scalable grading for large cohorts; effective feedback mechanisms

19	Sharma, P. & Singh – Personalized Job Search with AI: Skill-Based Matching	IJERT (2023)	NLP, deep learning, transformer models	Transformer-based matching outperforms rule-based; improves recruiter efficiency
20	Shivhare, K. et al. – ResumeCraft: ML-Powered Web Platform for Resume	IJRASET (2024)	ML models, web app (HTML/CSS/JS)	ATS-friendly resumes; improved user satisfaction

2.3 : Key Gaps in the Literature:

- Most tools available today concentrate on only one task - whether paperwork, cover letter writing or job role suggestions - meaning students find themselves using multiple platforms rather than one, unified system.
- Many resume builders still rely on very generic templates that hardly reflect a student's unique skills or meet the different requirements of various industries.
- Very few research studies have been carried out to find out and combine the three components of resume parsing, AI-generated content, and job role prediction into one integrated workflow.
- Some of the current systems are even failing when they are given resumes with a different formatting, scanned PDF, or creative designs, so they are less reliable.
- The research refers to cover-letter solutions that are mostly based on templates and do not generate content that is customized to the student's background.
- Job role recommendation from AI still is at a very basic level and there are very few studies that can link user skills with actual industry job frameworks.
- Interview-practice tools typically have a fixed set of questions and have little feedback, so they are not as useful for actually preparing.
- Many academic papers put a lot of emphasis on the performance of an algorithm, and the user experience and ease of use are often neglected.
- There is not much work being done with the intention of making these systems affordable, accessible, and especially student-friendly.
- There are few open source projects that make an effort to provide a complete career support for educational institutions to adopt
- Research prototypes are rarely concerned with issues such as cloud hosting, security, building scalable backends that can support real users.

CHAPTER 3: SYSTEM DEVELOPMENT

3.1 Requirements and Analysis:

The development of the platform was started with a clear understanding of what users are actually expecting from when they prepare for job applications. Most students or early-career professionals have difficulties with organizing their documents, keeping track of different job portals, keeping their responses of different quality across resumes, cover letters, and interviews. In order to make sure that the system addressed these issues in a meaningful way, the requirements were broken down into functional requirements, non-functional requirements, and user experience considerations.

Functional Requirements :

- **User Registration and Authentication** Opening a user account with authentication is the most fundamental function of a platform. Essentially, it is the developers' job to ensure that the users will have a safe environment to disclose their personal information. Obviously, people will probably be entering their personal and academic information therefore it is very important that the highest level of data privacy is kept at all times..
- **Resume Builder**: Users should have the capability to input educational qualifications, skills, job history, projects, and achievements in a well-structured manner. The system is obliged to verify the inputs and locate any missing or ambiguous parts.
- **Cover-Letter Help**: The medium should generate or upgrade cover letters based on job roles. It ought to be that the user edits the text and gets suggestions for improving the tone, giving more clarity, and being more relevant.
- **Interview Practice Module:** Users should be enabled to view the roles they could do and be interested in, given their skills, and keep track of the applications with statuses such as Applied, Shortlisted, or Rejected.
- **Job Recommendation and Tracking:** Users should be enabled to view the roles they could do and be interested in, given their skills, and keep track of the applications with statuses such as Applied, Shortlisted, or Rejected.

Non-Functional Requirements:

- **Performance:** The service should be such that it answers promptly even when one is accessing different modules one after another. To avoid user impatience, all API calls should be completed within a reasonable time.
- **Scalability**: The system should handle growing user data—résumés, interview attempts, and saved jobs—without affecting overall performance.
- **Security**: Passwords must be encrypted, and sensitive personal information must be stored securely. Unauthorized users should not be able to access private profiles.
- **Usability**: The interface should be simple enough for first-time users. Navigation has to feel natural, with clear labels and an uncluttered design.

3.2 Project Design and Architecture:

The project follows a simple and clear plan. Such a structure makes the system a piece of cake to change or add to later. In fact, there are one frontend part and one backend part of the architecture. Frontend mainly concentrates on user interface elements and interactions such as forms, dashboards, and resume previews. By contrast, backend generally serves as a helper layer to these functions by storing data, handling user profiles, keeping a record of job applications, generating results.

For example, we chose component-based architecture where each feature: authentication, job tracking, resume generation, etc. constitutes an independent component. Not only completing the code beautifully, this also allows for the change of any function without the influence of the rest. The design of the database schema associating user information with job applications and resume data integration hence improves the pipeline efficiency and reliability.

3.2.1 User Layer:

- **Inputs** Easily build your profile by entering your personal details and educational background.
- **Actions**:
    - Upload Resume / Profile Data
    - Attempt Coding Exam
    - Request Cover Letter

- Participate in Interview Simulation
- Job application Tracker

3.2.2 AI Processing Layer (Modules):

1. **Resume Analyzer (NLP + ML):**

    - Extracts keywords, skills, achievements.
    - Matches content with job descriptions (ATS optimization).
    - Suggests missing keywords and formatting fixes.

2. **Job Role Predictor (Recommendation Engine):**

    - Uses resume + skill data.
    - Suggests suitable future career roles.

3. **Cover Letter Generator (LLM):**

    - Uses resume + job description.
    - Generates tailored, ATS-friendly cover letter

4. **Job Application Tracker:**

    - Company name + role + application status + Application link
    - Generates tracker of different application of different portals in one place.

5. **AI Interviewer Dojo(LLM+NLP):**

    - Generates personalized interview questions.
    - Conducts mock interview through interactive chat.
    - Analyzes responses and provides feedback with improvement suggestions.

3.2.3 Output Layer:

- Generated Resume (ATS-Optimized)
- Cover Letter
- Suggested Career Roles
- AI Interview Feedback Report (Score, strengths, recommendations)

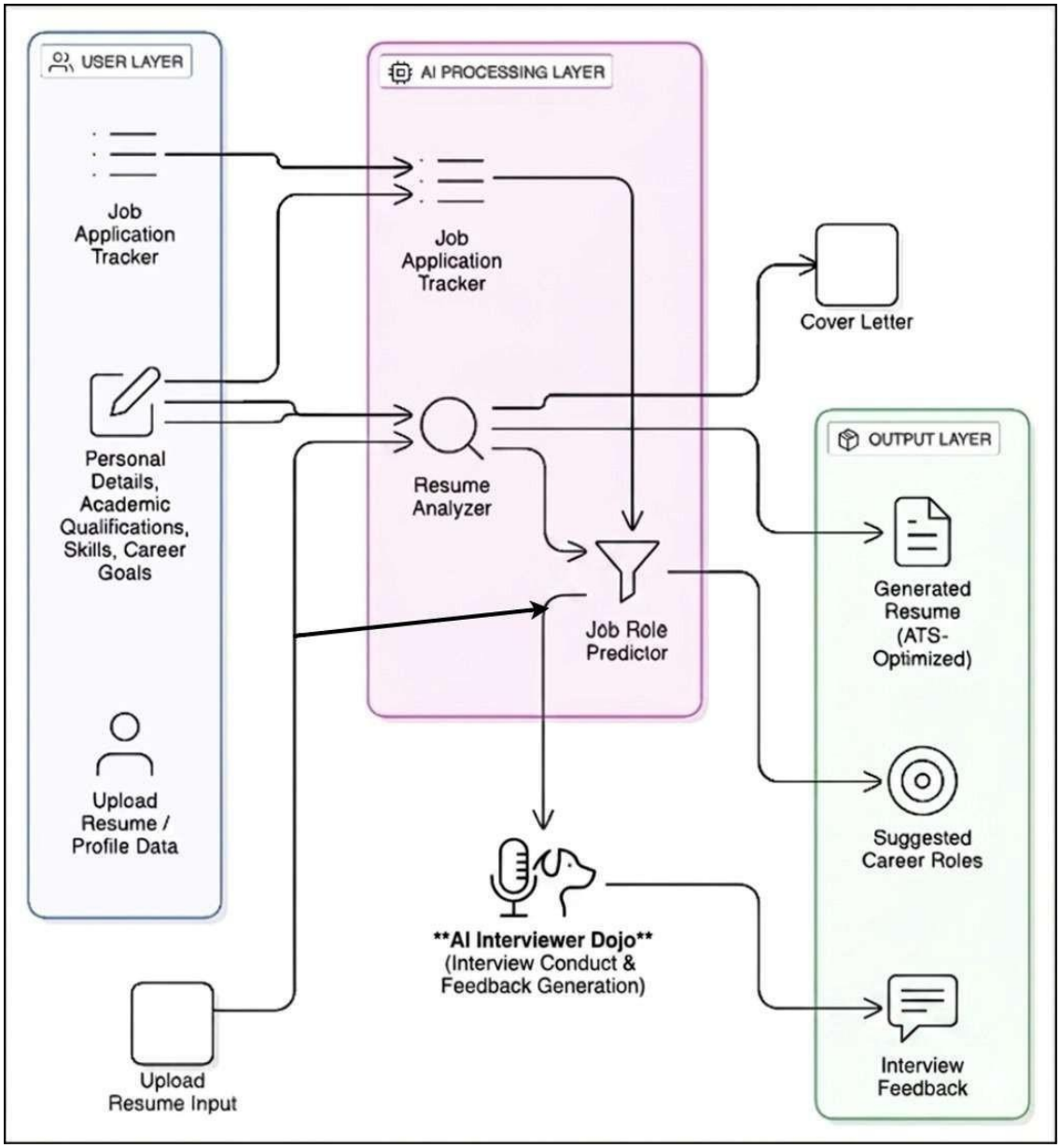

Figure 3.1: System Architecture of Career Companion

The software is very neatly organized into three layers:(*see figure 3.1*) the frontend specifies the user interface and interaction, and the backend implements the business logic authentication storage, and API. User accounts, resume data, and job applications are all stored in MongoDB. The separate modularization of each major feature provides the possibility for easy future upgrades without any problems. The API endpoint design follows the structure of layers of an onion, e.g. auth resume cover letter, job tracker, etc. Development of fully functional prototype of the project is the main objective of this semester, that's why the overall design of the architecture is fairly simple. Predictive analytics and advanced AI interfacing are features that will be included in the project's future plans.

3.3 Tools and Technologies:

We built this system using.

- **Frontend:** React.js / HTML / CSS
- **Backend:** Node.js or Django (depending on implementation)
- **Database:** MongoDB / MySQL
- **Resume Generation:** Template-driven PDF generation
- **API Integration:** RESTful API architecture
- **Authentication:** JWT / Session-based login
- **UI/UX Design:** Crisp and up-to-date design that is also adaptable to different device sizes
- **AI Integration:** Implemented the use of Grog API(LLaMA 3) and Google Generative AI for creating interview practice and feedback generation.
- **File Handling:** Multer for resume uploads
- **PDF Processing:** PDF-Parse for extracting resume text
- **HTTP Communication:** Axios for frontend-backend interaction
- **Build Tool:** Vite for fast development and optimized builds
- **Real-Time Interaction:** Conversational interview platform that can produce spontaneous answers
- **NLP Processing:** Employed in analyzing resumes, formulating questions and assessing feedback.

3.4 Data Preparation:

While developing my system, I double-checked that the parameters that really mattered to me were included, such as text skills education, audio files of spoken responses and the written answers in pre-formatted cover letter. As I faced numerous possibilities of inputs, I designed my system with a logical and airtight framework not only to avoid any confusion but also to provide the processing with a very smooth flow.

3.4.1 Collecting User Input:

Users have their own preferred style, both with their description and in the answers given to the system. While some may enter a long text, other may answer in a single sentence, or a couple of phrases. This variation is tackled by normalizing all the inputs.

3.4.2 Standardizing Resume Content

when users put together their resumes, invariably, there are areas and sections in the resume containing description gaps, and therefore the particular system must address the gaps by identifying:

- focusing on descriptions that are overly broad and words that are used in ways that are redundant.
- Highlight missing fields like dates, roles, or achievements
- Separate skills into clean categories

This ensures the résumé data can be used for later modules like job recommendations.

3.1.1 Preparing Cover-Letter Information

Cover letters require more natural writing, so the system:

- Breaks user text into paragraphs
- Checks for overly generic statements
- Identifies sentences that may need more context

The text is then restructured so the user can edit it smoothly.

3.1.2 Preparing Interview Responses

Interview answers are stored along with metadata such as:

- Question category
- Timestamp of the attempt
- Response length

This helps the system provide more meaningful feedback when the user returns to practice

3.1.3 Data Storage and Retrieval

All cleaned and structured data is stored in MongoDB. The schema flexibility allows users to update their information anytime without facing rigid form restrictions.

3.5 Ollama & Llama 3.x Model Architecture

- Ollama acts as the control layer for the Llama 3. x models, which are based on a more advanced Decoder-Only Transformer design. Differently from earlier models, Llama 3. x is focused on efficient running of inference through the use of Grouped-Query Attention (GQA). With this technique, several query heads are able to share a single key-value head, which greatly reduces the amount of memory bandwidth needed for the autoregressive decoding process. Ollama, upon loading a Llama 3. x model (usually in GGUF format), calls the llama. cpp backend to carry out these operations, enabling top-notch text generation even on devices with very limited VRAM by swapping parts of the neural network layers between the GPU and CPU.
- A major hallmark of the Llama 3. x design is its vastly larger Tokenizer, which boasts a vocabulary of 128,000 tokens (compared to 32,000 in Llama 2 Due to this enlargement, text coding becomes quite more efficient and is able to give top-notch quality outputs in diversified language as well as coding scenarios. On a technical level, the model depends on Rotary Positional Embeddings (RoPE) to carry positional details. Rather than simply adding vectors that represent absolute positions to the embeddings, RoPE rotates the matrix of the Query (Q) and Key (K) at each layer. For a position m and a vector x of dimension d,the equation for the transformation becomes:

$$f(x,m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

- This means that the attention scores will only be affected by the relative distance between tokens, and this way the model will be able to deal with the large context windows (up to 128k for Llama 3. 1) that Ollama supports. Instead of using the ordinary ReLU, the internal feed-forward network (FFN) of Llama 3. x has the SwiGLU (Sigmoid-Weighted Linear Unit) activation function, which allows gradients to flow better and gives a powerful nonlinear mapping. In a FFN layer the mapping from input x to output h is:

$$\mathrm{SwiGLU}(x, W, V, b, c) = (\mathrm{Swish}_1(xW + b)) \otimes (xV + c)$$

- Here, "⊗" means element-wise multiplication. To keep the numbers stable in its very deep layers (for example, 32 layers in the 8B model), Llama 3. x implements RMSNorm (Root Mean Square Layer Normalization). Compared to regular LayerNorm, RMSNorm

does not re-center the inputs (by removing the mean) but simply rescales them, which makes the computation less heavy:

$$\bar{a}_i = \frac{a_i}{\sqrt{\frac{1}{n}\sum_{j=1}^{n} a_j^2 + \epsilon}} \cdot \gamma_i$$

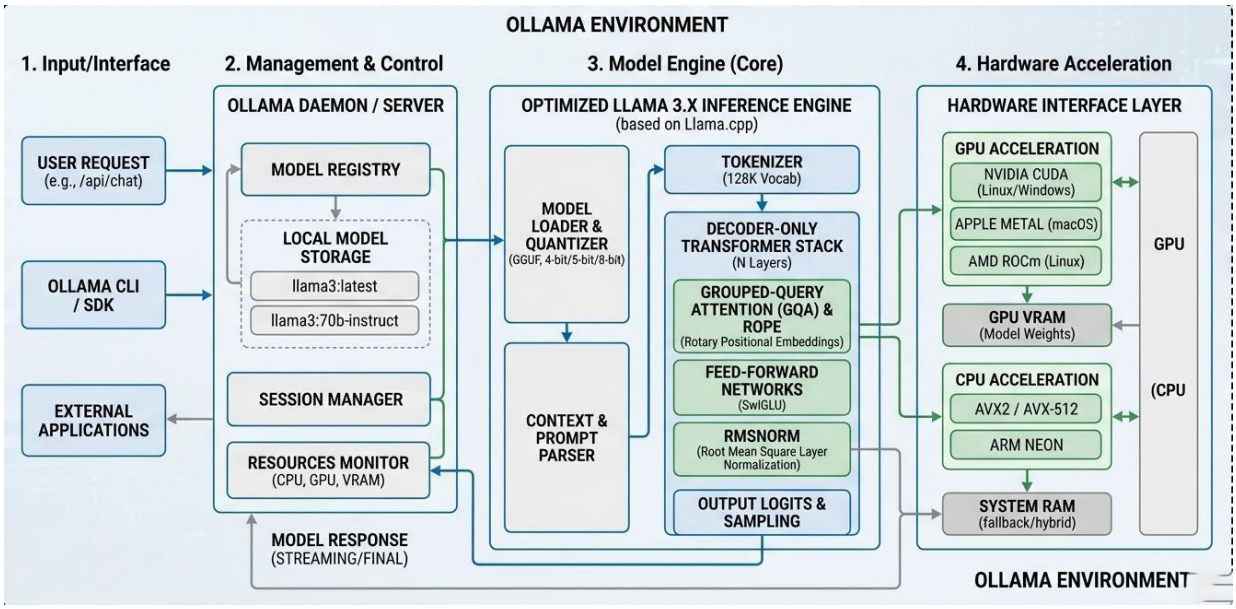

Figure 3.2: Llama 3.x Model Architecture

The image represents the modular setup of the Ollama environment (*see figure 3.2*), depicting the complete flow from client requests to hardware-accelerated inference. It shows the way the Ollama Daemon manages local model operations and at the same time uses a performance-tuned Llama 3. x engine to carry out sophisticated transformer functions running on GPU and CPU resources.

3.6 Implementation:

3.1.4 Workflow of Key Features:

- Resume Generator + Job Prediction: User fills form→ AI generates resume → Predicts future job roles (see Figure 3.2 Flowchart of Resume Generator with Job Prediction).
- **Cover Letter Generator:** User selects job → AI drafts personalized cover letter (*see Figure 3.3* Flowchart of Cover Letter Generator).
- **Job Application Tracker :** allows users to add and

manage their job applications by entering details like company, role, and application status

- **Ai Interviewer Dojo:** User uploads resume/profile → Resume is parsed and processed → AI generates personalized interview questions → System conducts mock interview → Responses are analyzed → AI provides feedback and improvement suggestions .

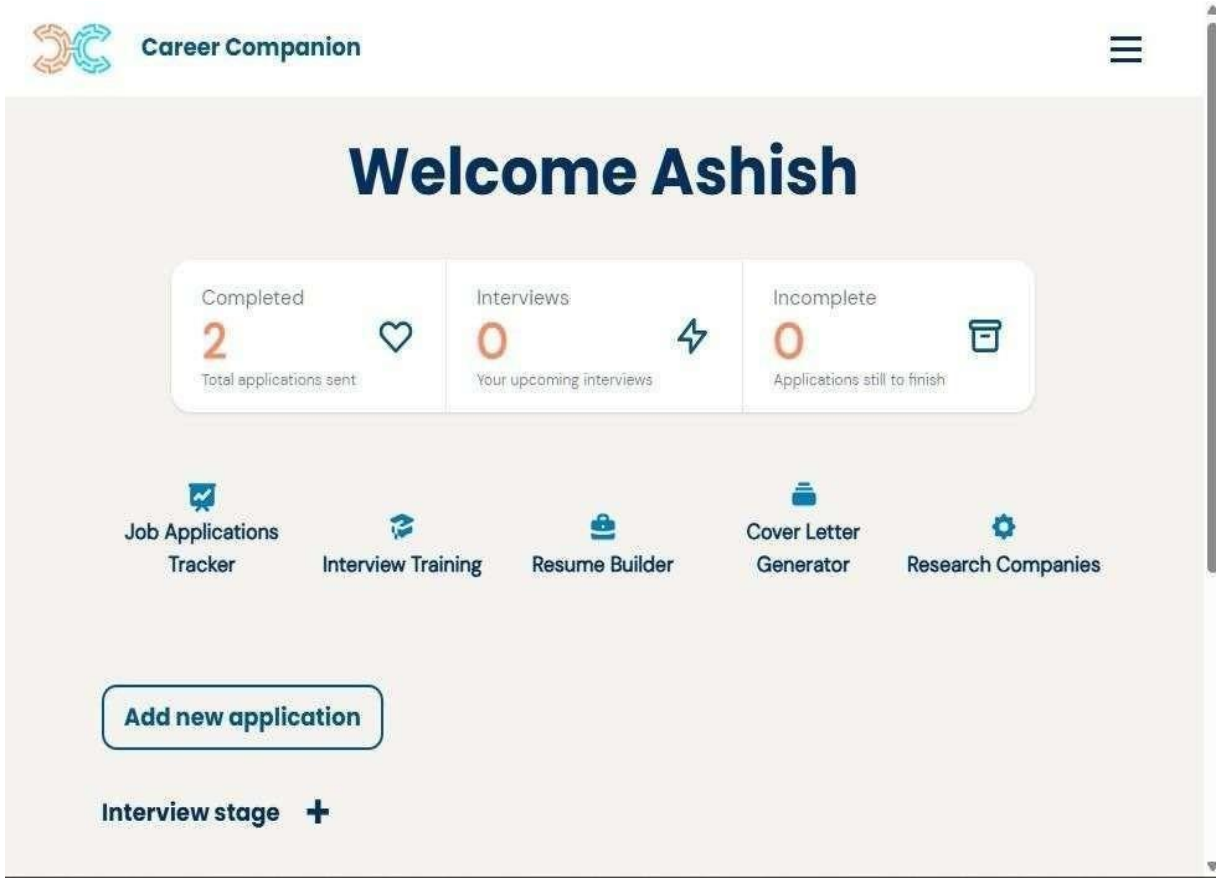

Figure 3.3: Website Frontend

This picture shows how actual website look like with all the features includes Job application tracker, resume builder ,research companies and so on.

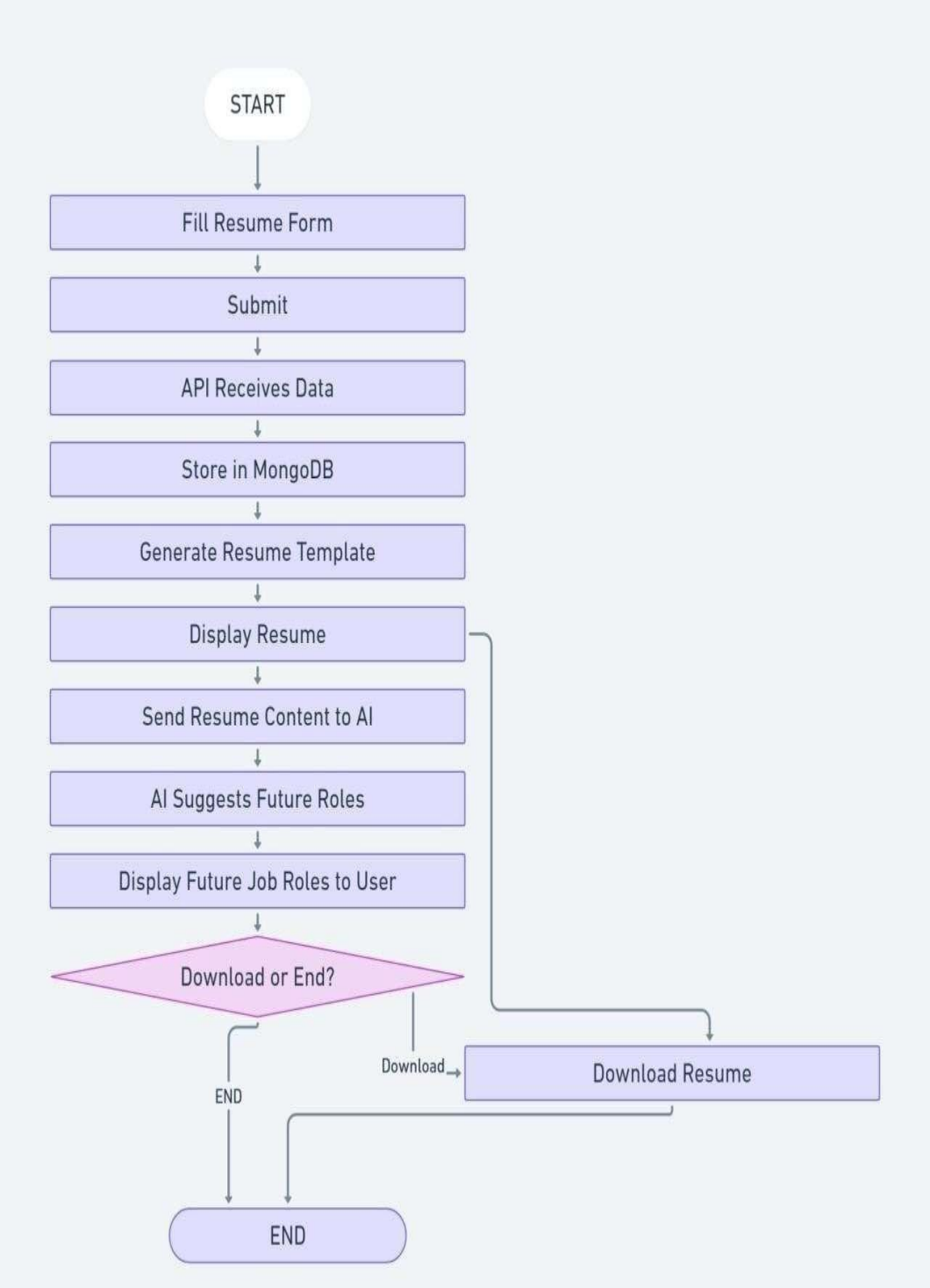

Figure 3.4: Flowchart of Resume Generator with Job Prediction

The flowchart illustrates how the user inputs personal and career details, which are processed by the AI to generate an ATS-optimized resume. Simultaneously, the system analyzes skills and experience to predict suitable future job roles, providing both a professional document and career guidance in one step.

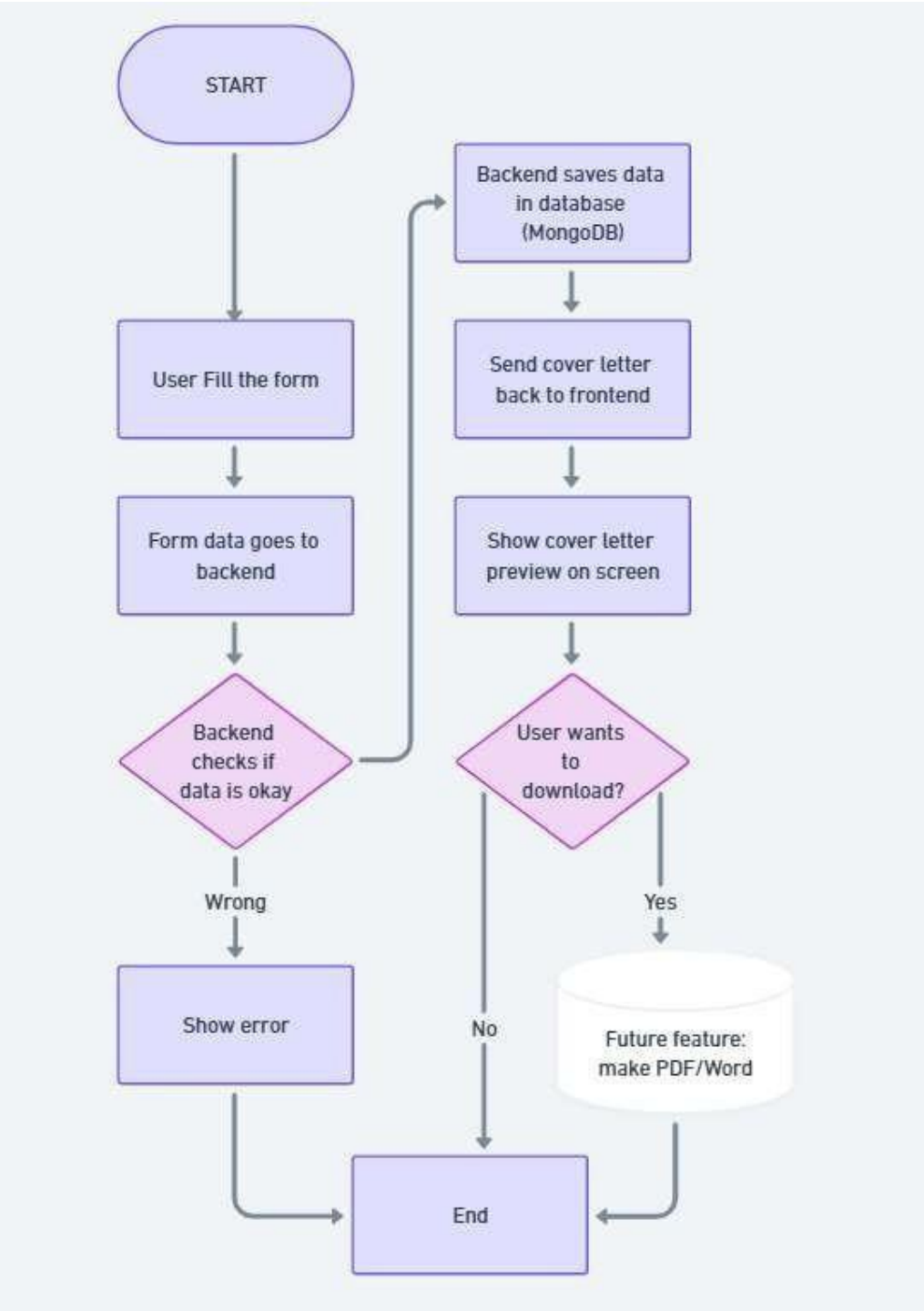

Figure 3.5: Flowchart of Cover Letter Generator

This flowchart illustrates the workflow for an online cover letter generator. A user submits a form, which the backend validates, saves to a MongoDB database, and then uses to generate a cover letter preview. The user can view this preview on the screen, with a future feature planned to allow downloading

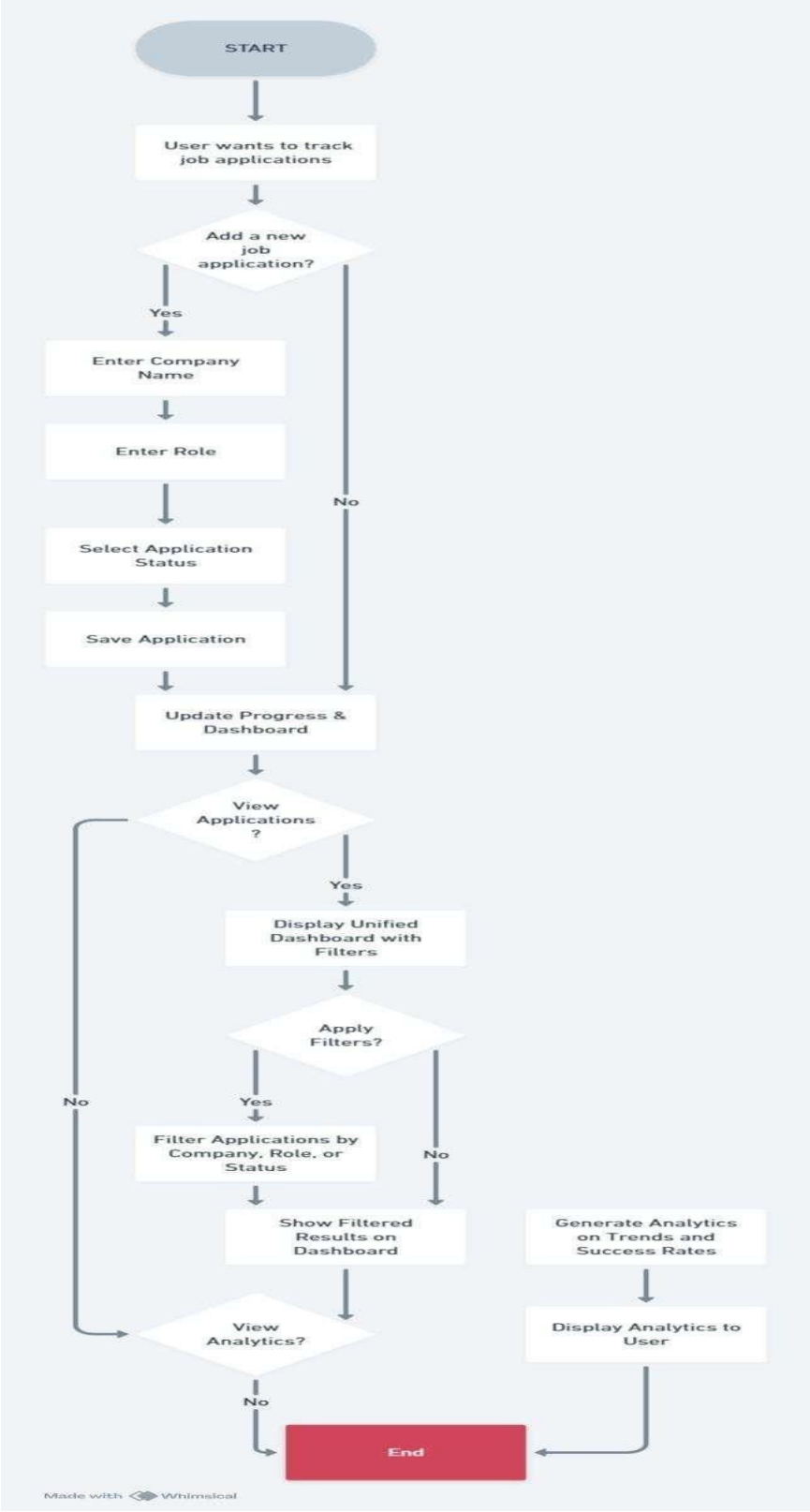

Figure 3.6: Flowchart of Job Application Tracker

The flowchart illustrates how users enter job application details, which the system validates and stores in the database. It also refreshes the dashboard so users can keep track of status changes.

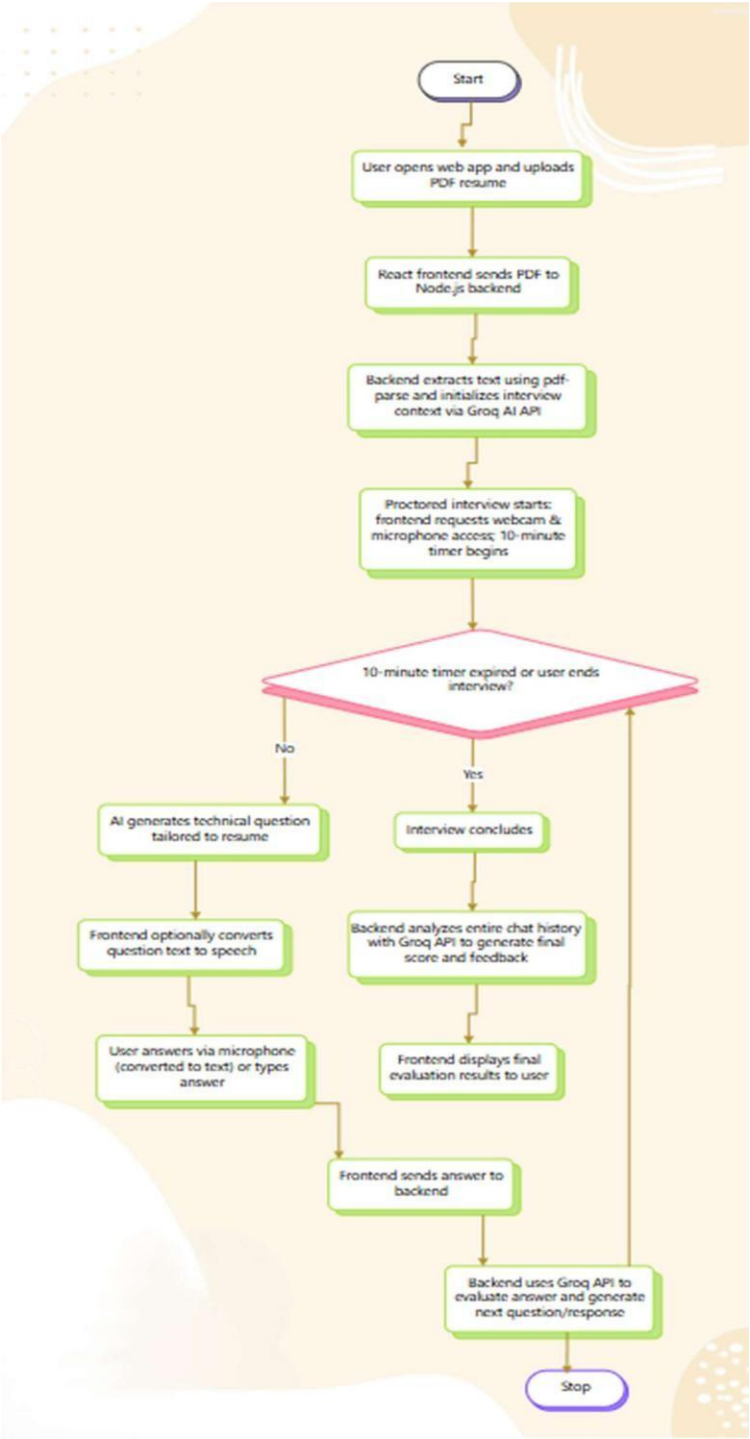

Figure 3.7: AI-Based Mock Interview System Workflow

This diagram illustrates the process behind an AI-led interview practice. The person sends their resume to the software so the timed interview can start. The questions from the AI will be based on the resume, and a user may either speak or write their answers. After the interview ends, the program analyzes all input and issues a final score together with comments.

3.1.5 Code Snippets:

```
Career Companion > frontend > src > components > questionsForm > JS QuestionsForm.js > QuestionsF
import "./QuestionsForm.css";

function QuestionsForm(props) {
  return (
    <>
      <form onSubmit={props.handleAnswersSubmit}>
        <h3>Question {props.questions[0]}?</h3>
        <textarea
          value={props.answer1}
          onChange={props.handleAnswer1Change}
          className="w-full md:w-4/5  h-20 border-2 border-gray-100 shadow
        ></textarea>

        <h3>Question {props.questions[1]}?</h3>
        <textarea
          value={props.answer2}
          onChange={props.handleAnswer2Change}
          className="w-full md:w-4/5  h-20 border-2 border-gray-100 shadow
        ></textarea>

        <h3>Question {props.questions[2]}?</h3>
        <textarea
          value={props.answer3}
          onChange={props.handleAnswer3Change}
          className="w-full md:w-4/5  h-20 border-2 border-gray-100 shadow
        ></textarea>
```

Figure 3.8: Question Form Component in Frontend

Essentially, the React component code is given after you work out how questions are dynamically presented to users one after another, with each question corresponding to a controlled textarea for the user's real-time input and live validation.

Here, handling the form gets extremely simple, plus you can use it again and again, which harmonizes nicely with the modular style of the frontend, basically just a natural result.

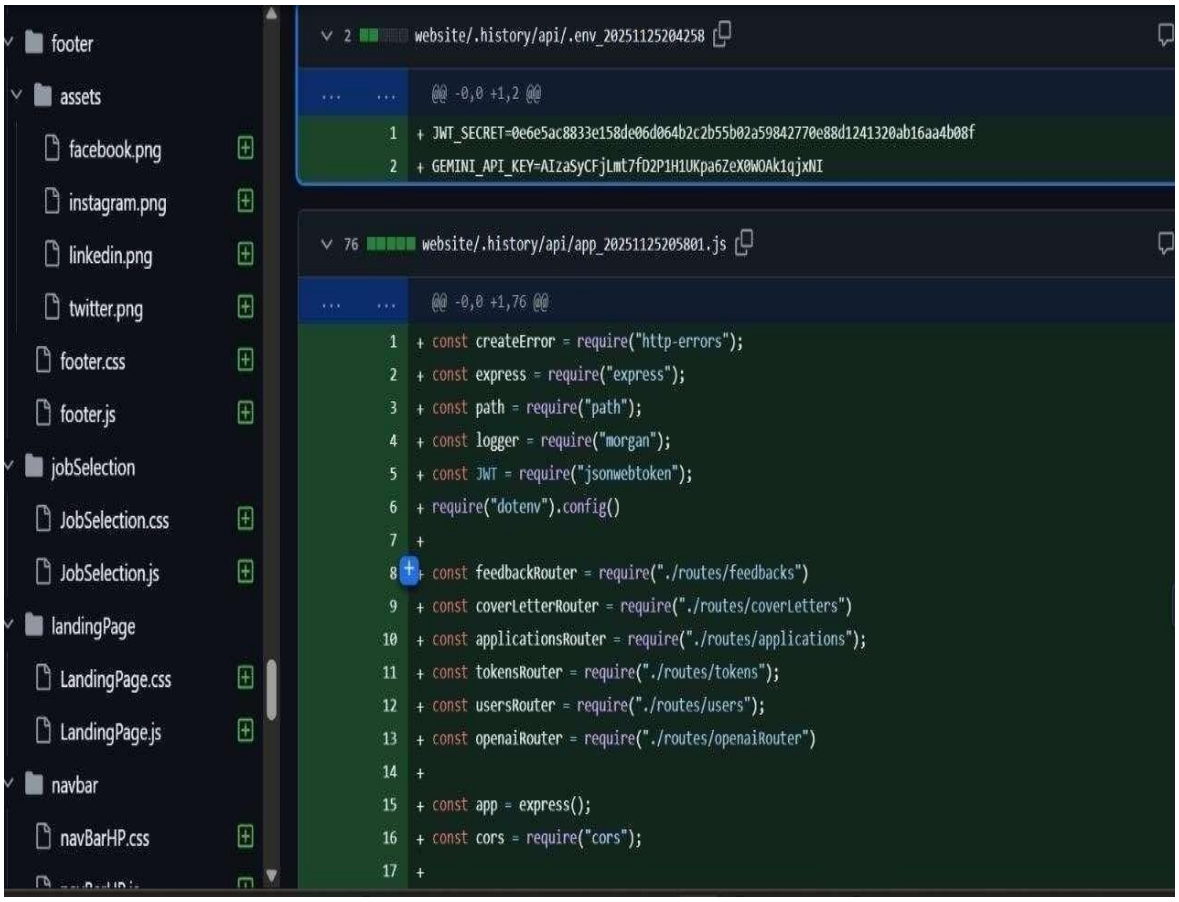

Figure 3.9: Backend Environment Variable and Router Configuration

Looking at the Node. js backend side, environment variables, keys for APIs, and routes have been set up, and one can see that feedback, applications, and cover letters are connected via Express routers .This modular structure not only promotes separation of features but also makes the backend easier to handle.

Career Companion > frontend > src > index.js > ...

```
1   import React from 'react';
2   import ReactDOM from 'react-dom/client';
3   import './index.css';
4   import App from './components/app/App';
5   import reportWebVitals from './reportWebVitals';
6   import { BrowserRouter } from 'react-router-dom';
7   import './components/navbar/navBarLP.css';
8   import { disableReactDevTools } from '@fvilers/disable-react-devtools';
9
10  if (process.env.NODE_ENV === "production") {
11    window.BACKEND_API_SERVER_ADDRESS = "https://career-companion-0vnx.onren
12    disableReactDevTools();
13  } else {
14    window.BACKEND_API_SERVER_ADDRESS = "http://localhost:8080"
15  }
16
17  const root = ReactDOM.createRoot(document.getElementById('root'));
18  root.render(
19    <BrowserRouter>
20      <React.StrictMode>
21        <App />
22      </React.StrictMode>
23    </BrowserRouter>
24  );
```

Figure 3.10: Frontend Entry File and API Configuration

In fact, the frontend depicts very minimal involvement in the decision-making process of the backend API selection for a React app. It is an automatic setup: based on what environment the app is in - dev or production, the API is chosen without any intervention to the results of the app release being a clear indication of the correct performance of routing, rendering, and API communication of the app.

cv parser > preprocess_cv.py > ...

```python
# Libraries
import os
import re

import fitz
from docx import Document

from config import CV_FILES_DIR, EXTRACTED_TEXT_DIR
from logger import time_function

@time_function
def convert_docx_to_pdf(docx_path):
    """
    Converts a .docx file to .pdf. Requires Microsoft Word and works on Windows.
    Alternatively, can use unoconv for Linux/macOS or a cloud API.
    For simplicity and consistency with existing win32com usage, we'll use a similar approach if available.
    A more robust cross-platform solution would involve LibreOffice/unoconv or third-party libraries.
    """
    try:

        from docx2pdf import convert
        pdf_path = os.path.splitext(docx_path)[0] + '.pdf'
        convert(docx_path, pdf_path)
        print(f" [DOCX to PDF] Successfully converted {docx_path} to {pdf_path}")
        return pdf_path
    except ImportError:
        print(" [DOCX to PDF ERROR] 'docx2pdf' library not found. Please install it: pip install docx2pdf")
        print(" [DOCX to PDF INFO] Attempting conversion using win32com.client (Windows only)...")
        try:
            import win32com.client
            word = win32com.client.Dispatch("Word.Application")
            word.Visible = False
            doc = word.Documents.Open(docx_path)
            pdf_path = os.path.splitext(docx_path)[0] + '.pdf'
            # 17 = wdFormatPDF
            doc.SaveAs(pdf_path, FileFormat=17)
            doc.Close()
            word.Quit()
            print(f" [DOCX to PDF] Successfully converted {docx_path} to {pdf_path} using win32com.")
            return pdf_path
        except Exception as e:
            print(f" [DOCX to PDF ERROR] Failed to convert {docx_path} to PDF using win32com: {e}")
            return None
    except Exception as e:
        print(f" [DOCX to PDF ERROR] Failed to convert {docx_path} to PDF: {e}")
        return None
```

Figure 3.11: preprocess_cv.py (DOCX to PDF Conversion)

When converting .docx files to PDFs, our script makes use of the docx2pdf library. On the other hand, for Windows-based conversions, it depends on win32com,. and it also has error handling to verify that resumes are converted correctly before text extraction.

cv parser > regex_parser.py > ...

```python


def extract_contact_info(text):
    contact_info = {"email": None, "phone_numbers": [], "urls": []}

    # Email
    email_matches = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', text)
    if email_matches:
        unique_emails_ordered = sorted(list(set(email_matches)), key=text.find)
        if unique_emails_ordered:
            contact_info["email"] = unique_emails_ordered[0]

    # Phone Numbers
    phone_matches = re.findall(
        r'(?:\+?\d{1,3}[-.\s]?)?\(?\d{2,4}\)?[-.\s]?\d{3,4}[-.\s]?\d{4,6}\b',
        text
    )

    valid_phones = []
    for phone in set(phone_matches):
        normalized_phone = re.sub(r'[()\s.-]', '', phone)
        if len(normalized_phone) >= 10 and len(normalized_phone) <= 15 and normalized_phone.replace('+', ''),
            if len(normalized_phone) == 10 and not normalized_phone.startswith(('+', '0')):
                normalized_phone = '+' + normalized_phone
            elif len(normalized_phone) == 11 and normalized_phone.startswith('0'):
                normalized_phone = '+91' + normalized_phone[1:]
            valid_phones.append(normalized_phone)
    contact_info["phone_numbers"] = list(set(valid_phones))

    # ✅ Correct URL Extraction
    url_matches = re.findall(r'https?://[^\s)>\]"]+', text)
    contact_info["urls"] = list(set(url_matches))

    return contact_info

def chunk_text(text, max_chunk_size=1500, overlap=80):
    """
    Chunks text into smaller pieces with overlap for RAG.
    Prioritizes splitting at natural boundaries (e.g., double newlines).
    """
    chunks = []
    paragraphs = text.split('\n\n') # Split by double newlines (paragraphs)
    current_chunk = ""

    for para in paragraphs:
```

Figure 3.12: regex_parser.py (Contact Info & Chunk Extraction)

This piece of code illustrates the regex parser that I use for extracting contact-type information such as email addresses, phone numbers, URLs, and other pieces of data from resume text. It also sanitizes and normalizes phone numbers and divides the text into chunks for the later stage of lln. This code is instrumental in obtaining your information from your resume text in an understandable manner.

```
cv parser > llm_parser.py > ...
# llm_parser.py
import ollama
import json
import re
from config import OLLAMA_HOST, OLLAMA_MODEL_NAME, OLLAMA_EMBEDDING_MODEL_NAME
from logger import time_function
# Initialize Ollama client
client = None
try:
    client = ollama.Client(host=OLLAMA_HOST)

except Exception as e:
    print(f"Error connecting to Ollama at {OLLAMA_HOST}: {e}")
    print("Please ensure Ollama is running and the specified model is downloaded.")
    client = None # Set client to None if connection fails


@time_function
def _call_ollama(prompt, model=OLLAMA_MODEL_NAME, context=None):
    """Helper function to call Ollama model with error handling."""
    if client is None:
        return None # Return None if client wasn't initialized

    messages = [{"role": "user", "content": prompt}]
    if context:
        # Prepend context as a system message for better instruction following
        messages.insert(0, {"role": "system", "content": f"Here is the relevant text context to extract infor

    try:
        response = client.chat(
            model=model,
            messages=messages,
            stream=False # We want the full response
        )
        return response['message']['content']
    except Exception as e:
        print(f"Error calling OLLAMA ({model}) at {OLLAMA_HOST}: {type(e).__name__}: {e}")
        return None


def _parse_llm_json_output(llm_output):
    """
    Robustly extracts the first valid JSON block (array or object) from the LLM output.
    Handles extra text, markdown, multiple JSON snippets.
    """
    if not llm_output:
```

Figure 3.13: llm_parser.py (LLM-Based Resume Processing)

This use case illustrates how this function forces the LLM to scan and extract detailed information from resumes. It also demonstrates how to use prompt, set context, and obtain answers via the Ollama client. The code sends prompts in a determined manner and returns pure resume data.

cv parser > config.py > ...

```python
# config.py
import os

# Base directory for the project
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# Directory where raw CV files are stored (PDF, DOCX)
CV_FILES_DIR = os.path.join(BASE_DIR, 'cv_files')

# Directory where preprocessed plain text from CVs will be saved
EXTRACTED_TEXT_DIR = os.path.join(BASE_DIR, 'extracted_text')

# Directory where final parsed JSON results will be saved
REGEX_PARSED_RESULTS_DIR = os.path.join(BASE_DIR, 'parsed_results')

# Ensure directories exist
os.makedirs(CV_FILES_DIR, exist_ok=True)
os.makedirs(EXTRACTED_TEXT_DIR, exist_ok=True)
os.makedirs(REGEX_PARSED_RESULTS_DIR, exist_ok=True)

# Ollama Configuration
OLLAMA_HOST = "http://localhost:11434"
OLLAMA_MODEL_NAME = "llama3.2:latest"
# OLLAMA_MODEL_NAME = "llama3.3-32k:latest"
OLLAMA_EMBEDDING_MODEL_NAME = "mxbai-embed-large"  # mxbai-embed-large:334m, mxbai-embed-large:latest (335M),



# mxbai-embed-large:335m, mxbai-embed-large:latest (334M), nomic-embeded-text (137M)
```

Figure 3.14: config.py (System Configuration & Directory Setup)

This figure shows the config file with projects folders, storage, environment and more. It also display setup for Ollama model names and embedings models parsing the content.

cv parser > resume_ai_analysis.py > ...

```python
import json
from llm_parser import _call_ollama, _parse_llm_json_output
from config import OLLAMA_MODEL_NAME

def analyze_resume_with_llm(parsed_resume):
    """
    Uses Ollama LLM to generate:
    - Career Growth Potential (0-10)
    - ATS Compatibility (0-100)
    - Recommended Jobs
    Returns a dictionary with all results.
    """

    # Prepare structured input for the LLM
    resume_json = json.dumps(parsed_resume, indent=2)

    prompt = f"""
    You are an expert career and recruitment assistant.

    Below is a candidate's structured resume data in JSON format.
    Analyze it carefully and provide:
    1. A **Career Growth Potential Score** (0-10) based on experience, education, and skills.
    2. An **ATS Compatibility Score** (0-100) — evaluate section completeness, keyword diversity, and structu
    3. A **list of 3-5 recommended job roles** that best match the candidate's profile.
    4. A **short summary (2 sentences)** explaining your reasoning for both scores.

    Return your answer as a clean JSON object exactly like this format:
    ```json
    {{
      "career_growth_score": 8.5,
      "ats_score": 77,
      "recommended_jobs": ["Python Developer", "Data Analyst", "ML Engineer"],
      "summary": "Strong technical base with good experience. Resume could use better keyword optimization."
    }}
    ```

    Candidate Resume JSON:
    {resume_json}
    """

    response = _call_ollama(prompt, model=OLLAMA_MODEL_NAME)
    parsed = _parse_llm_json_output(response)

    if not parsed or not isinstance(parsed, dict):
        return {
            "career_growth_score": "N/A",
            "ats_score": "N/A"
```

Figure 3.15: Resume Analysis Module Using LLM

In this picture is the Python program that makes the resume data and types it to an LLM for review. The prompt template shows how career growth, ATS score and job advice are made. It is the main engine of the AI resume review feature.

```python

# --- Main Parsing Pipeline ---

@time_function # Apply the decorator here
def parse_cv_with_pipeline(file_path):
    performance_logger.info(f"Processing: {os.path.basename(file_path)}")
    parsed_data = {
        "file_name": os.path.basename(file_path),
        "name": None,
        "contact_info": {"email": None, "phone_numbers": [], "urls": []}, # Initialize to ensure keys exist
        "skills": [],
        "experience": [],
        "education": [],
        "projects": [],
        "certifications": [],
        "languages": []
    }

    with open(file_path, 'r', encoding='utf-8') as f:
        raw_text_content = f.read()

    clean_text_content = clean_text_for_parsing(raw_text_content) # Assuming text is already preprocessed

    # --- Step 1: Initial Regex Extraction (Name, Contact Info) ---
    # Name - Try LLM first for better accuracy
    name_query = "What is the full name of the candidate in this resume?"
    # Pass a reasonable portion of the text where the name is likely found
    name_llm = extract_name_with_llm(clean_text_content[:2000]) # Increased context for name
    if name_llm and name_llm.strip() != "N/A":
        parsed_data["name"] = name_llm.strip()
        performance_logger.info(f"    Name (LLM): {parsed_data['name']}")
    else:
        # Fallback regex if LLM fails (less accurate, but a backup)
        # Tries to find capitalized words at the beginning, usually a name
        name_match = re.search(r'^[A-Z][a-z]+(?:\s[A-Z][a-z]+){1,3}', clean_text_content[:500])
        if name_match:
            parsed_data["name"] = name_match.group(0).strip()
            performance_logger.info(f"    Name (Regex Fallback): {parsed_data['name']}")
        else:
            parsed_data["name"] = "N/A"
```

Figure 3.16: Main Parsing Pipeline Initialization

This code shows the parse_cv_with_pipeline function that starts processing a resume file. It begins by creating the main data object (parsed_data) with empty fields for skills, experience, and education. It then performs the first extraction pass, trying to find the candidate's name first with an LLM, then with a regex fallback, along with contact info.

```
cv parser > regex_parser.py > ...
247  def parse_cv_with_pipeline(file_path):
291
292      # --- Step 2: RAG Pipeline for other sections ---
293      # Generate chunks and embeddings for the entire document once
294      chunk_texts = chunk_text(clean_text_content)
295      # Filter out any None embeddings
296      # chunk_embeddings_start_time = time.time()
297      # chunk_embeddings = [get_embedding(chunk) for chunk in chunk_texts]
298      # chunk_embeddings_filtered = [embed for embed in chunk_embeddings if embed is not None]
299      # chunk_texts_filtered = [chunk_texts[i] for i, embed in enumerate(chunk_embeddings) if embed is not None
300      # chunk_enbedding_end_time = time.time()
301      # total_time = chunk_enbedding_end_time - chunk_embeddings_start_time
302      # performance_logger.info(f"Total get_embedding execution time: {total_time:.4f} seconds")
303      # performance_logger.info(f"Generating embeddings for {len(chunk_texts_filtered)} usable chunks...")
304
305
306      # Skills
307      skills_query = "List of distinct technical skills, programming languages, software, tools, and methodolog
308      # Corrected call: removed redundant clean_text_content argument
309      # skills_chunks = retrieve_relevant_chunks(clean_text_content,skills_query, chunk_embeddings_filtered, ch
310      # performance_logger.debug(f"Retrieved {len(skills_chunks)} chunks for skills. Context length: {len(' '.j
311      # performance_logger.debug(f"Context for Skills (first 800 chars):\n{' '.join(skills_chunks)[:800]}...") 
312      parsed_data["skills"] = extract_skills_with_llm(' '.join(chunk_texts)) # Changed to use all filtered chun
313      performance_logger.info(f"    Skills (LLM via RAG): {len(parsed_data['skills'])} entries")
314
315      # Experience
316      experience_query = "Candidate's work experience, employment history, EMPLOYMENT RECORD RELEVANT TO THE AS
317      # Corrected call: removed redundant clean_text_content argument
318      # experience_chunks = retrieve_relevant_chunks(clean_text_content,experience_query, chunk_embeddings_filt
319      # performance_logger.debug(f"Retrieved {len(experience_chunks)} chunks for experience. Context length: {l
320      # performance_logger.debug(f"Context for Experience (first 1500 chars):\n{' '.join(experience_chunks)[:10
321      parsed_data["experience"] = extract_experience_with_llm(' '.join(chunk_texts))
322      performance_logger.info(f"    Experience (LLM via RAG): {len(parsed_data['experience'])} entries")
323
324      # Projects
325      projects_query = "List of projects, assignments, or key deliverables with descriptions, technologies, cli
326      # Corrected call: removed redundant clean_text_content argument
327      # projects_chunks = retrieve_relevant_chunks(clean_text_content, projects_query, chunk_embeddings_filtere
328      # performance_logger.debug(f"Retrieved {len(projects_chunks)} chunks for projects. Context length: {len('
329      parsed_data["projects"] = extract_projects_with_llm(' '.join(chunk_texts))
330      performance_logger.info(f"    Projects (LLM via RAG): {len(parsed_data['projects'])} entries")
331
332      # Certifications (Prioritize section extraction, fallback to RAG)
333      certifications_section_text = extract_section(clean_text_content, "CERTIFICATIONS")
334      if not certifications_section_text: # If "CERTIFICATIONS" not found, try "TRAINING"
335          certifications_section_text = extract_section(clean_text_content, "TRAINING")
336
```

Figure 3.17: RAG-Powered Section Extraction

This part shows how RAG works step by step: Chunking and creating embeddings for the document content.. It shows how we find the parts (Skills, Experience, Projects) we want by running a similarity search to get the parts we need.. The last parts that make sense now get sent to an LLM (extract_skills, extract_experience, etc.) and it makes structured, parsed data for each part.

```python
# ---- Text Cleaning Function ----
def clean_text(text):
    """
    Performs cleaning on extracted text to remove artifacts and normalize formatting.
    """
    if not isinstance(text, str):
        return ""
    # Normalize line endings
    text = re.sub(r'\r\n|\r','\n',text)
    # Remove common OCR artifacts
    text =re.sub(r'(^|\s)e@(\s|$)', ' ', text)
    text = text.replace('e@', '')
    # Replace specific problematic characters
    text = text.replace('¢', ' ').replace('«', ' ').replace('*', ' ')
    text = text.replace('©', '').replace('™', '').replace('®', '')
    text = text.replace('@', '')
    text = text.replace('|', ' ')
    text = text.replace('_', ' ')
    text = text.replace('–', '-')
    text = text.replace('‘', "'").replace('’', "'")
    text = text.replace('“', '"').replace('”', '"')
    text = text.replace('…', '...')
    # Remove bullet points and similar symbols
    text = re.sub(r'[•▪○●✓►→]', ' ', text)
    text = text.replace('\f','')
    text = re.sub(r'[ \t]+', ' ', text)
    text = re.sub(r'\n{3,}', '\n\n', text) # Reduce excessive newlines
    # Strip leading/trailing whitespace from each line
    text = '\n'.join([line.strip() for line in text.split('\n')])
    return text.strip()
```

Figure 3.18: clean_text Preprocessing Function

The function clean_text(text) is a helper function that aims to make the raw text coming from the document more manageable. It employs pattern matching to unify line ends, remove typical OCR issues like extra @ and symbols, and replace complicated characters that are not standard. Finally, it removes bullet points, excessive new lines, and any spaces on the sides to make the text cleaner for the parsing step.

```
server.js   test-models.js   InterviewChat.jsx   README.md   .env
backend > server.js > app.post('/api/feedback') callback > prompt

// Endpoint for interview chat
app.post('/api/chat', async (req, res) => {
    try {
        const { conversationHistory, resumeText } = req.body;

        if (!conversationHistory || conversationHistory.length === 0) {
            return res.status(400).json({ error: 'Conversation history is required' });
        }

        // Construct the prompt with context
        const prompt = `You are an expert technical interviewer conducting an interview.
        Here is the candidate's resume for context:
        ---
        ${resumeText}
        ---
        Here is the conversation history so far:
        ${conversationHistory.map(m => `${m.role === 'interviewer' ? 'Interviewer' : 'Candidate'}: ${m.text}`).join('\n\n')}

        You are the Interviewer. Respond naturally to the Candidate's last message.
        Limit your response to 4-5 sentences and ask one clear question or follow-up at the end.
        Do NOT write "Interviewer:" at the start of your response, just provide the spoken text.`;

        const chatCompletion = await groq.chat.completions.create({
            messages: [{ role: "user", content: prompt }],
            model: "llama-3.3-70b-versatile",
        });

        res.json({ reply: chatCompletion.choices[0].message.content });
    } catch (error) {
        console.error('Error in chat:', error);
```

Figure 3.19: Handle Interview Chat Function

The function deals with AI-driven interview communication by accepting conversation history and resume as input. First of all, it validates the data and makes a prompt out of them which is an instruction for ai to imitate a technical interviewer. Also, the function guarantees that the answer is brief and at the same time contains a follow-up question that maintains the flow. Then it sends the prompt to language model, gets the answer and returns it to the frontend that makes the interview interactive and smooth.

```
server.js   test-models.js   InterviewChat.jsx   README.md   .env
backend > server.js > app.post('/api/feedback') callback > [0] prompt

// Endpoint to upload resume and initialize interview
app.post('/api/upload-resume', upload.single('resume'), async (req, res) => {
    try {
        if (!req.file) {
            return res.status(400).json({ error: 'No resume file uploaded' });
        }

        // Parse PDF
        const data = await pdfParse(req.file.buffer);
        const resumeText = data.text;

        // Ask Groq to analyze resume and generate the first interview question
        const prompt = `You are an expert technical interviewer. I will provide you with a candidate's resume.
        Analyze the resume and prepare to conduct a technical interview.
        Start by greeting the candidate, briefly summarizing their key strengths as you see them, and then ask the very first interview question based on their experience or skills.

        Resume text:
        ${resumeText}`;

        const chatCompletion = await groq.chat.completions.create({
            messages: [{ role: "user", content: prompt }],
            model: "llama-3.3-70b-versatile",
        });
        const initialMessage = chatCompletion.choices[0].message.content;

        res.json({
            resumeText: resumeText, // Send back so frontend can hold it in state if needed
            initialMessage: initialMessage
        });
    } catch (error) {
        console.error('Error processing resume:', error);
        res.status(500).json({ error: 'Failed to process resume' });
    }
});
```

Figure 3.20: Interview Setup Function

This function kick-starts the interview process by handling the receipt of resumes and extraction of relevant data. It first obtains the PDF file, checks if the file actually exists, and uses a parser to fetch the plain text from the document. Subsequently, the text is fused into a custom prompt where the AI is asked to analyze the candidate's profile, point out the strengths, and formulate the first interview question. This prompt is sent to the language model, which sends back an answer. Then the answer and the extracted resume text are given to the frontend to initiate the interview session.

```
import React, { useState, useEffect, useRef, useCallback } from 'react';
import { Send, User, Bot, CheckCircle, Mic, Camera, Clock } from 'lucide-react';
import axios from 'axios';
import ReactMarkdown from 'react-markdown';

const InterviewChat = ({ resumeText, initialMessage, onFinish }) => {
  const [messages, setMessages] = useState([
    { role: 'interviewer', text: initialMessage }
  ]);
  const [inputValue, setInputValue] = useState('');
  const [isTyping, setIsTyping] = useState(false);
  const [isEnding, setIsEnding] = useState(false);

  // Timers and states
  const [timeLeft, setTimeLeft] = useState(900); // 15 minutes overall
  const [answerTimeLeft, setAnswerTimeLeft] = useState(120); // 2 minutes per answ
  const [isListening, setIsListening] = useState(false);
  const [cameraActive, setCameraActive] = useState(false);
  const [isInterviewerSpeaking, setIsInterviewerSpeaking] = useState(false);

  const chatHistoryRef = useRef(null);
  const videoRef = useRef(null);
  const recognitionRef = useRef(null);
  const hasSpokenInitialRef = useRef(false);
  const prevInputRef = useRef('');

  // Setup Speech Recognition
  useEffect(() => {
    const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecog
    if (SpeechRecognition) {
      const recognition = new SpeechRecognition();
      recognition.continuous = true;
      recognition.interimResults = true;

      recognition.onresult = (event) => {
        let currentTranscript = '';
        for (let i = 0; i < event.results.length; i++) {
          currentTranscript += event.results[i][0].transcript;
        }
```

Figure 3.21: Interview Chat Function

InterviewChat component handles main frontend segments on interactive AI interview. It employs React state hooks for holding conversation state, user input, and system responses. Besides, it is in charge of typing indicators, interview timers, and session control purposes, etc. It uses speech recognition for voice input. References are used for easy management of chat history, audio, and video elements. Originally, the component dispatches a fixed message to set up the interview and keeps on updating the conversation by passing user inputs through the backend and exposing AI-created responses. By features such as microphone control, camera activation, and time tracking an interview feel is further enhanced. So, the component offers an efficient, interactive, and easy to use platform for conducting AI based technical interviews.

3.7 Key Challenges:

- Creating a system is a complicated journey during which many hiccups happen. Through enforcing stringent validation rules and providing users with UI messages that direct them, one significant problem was completely avoided.
- The next hurdle was to develop a resume maker compatable with several templates. Also, the concern was to avoid design chaos if a user enters a very long project description or a large number of skills. As they say, a picture is worth a thousand words and with the help of several images, we understood that we need to rework the template.
- One of our main concerns was the job tracker UI. While keeping the interface simple, allowing the users to change statuses with just a few clicks was quite a challenge. The only way to succeed was to minimize the job entry form and simplify the status options.
- Apart from that, we experienced some technical problems such as API failure, front and back communication errors, and also formatting issues while generating the PDF. We came up with the solution after a round of debugging, tweaking the code a bit, and going through the test cycles again and again. In fact, we managed to change every problem into a problem-solving opportunity thus the final solution works better and is more dependable.

CHAPTER 4: TESTING

4.1 Testing Strategy :

The pace of a well-planned testing approach was maintained constant through the entire development to guarantee proper operation of the system and reaching the goal features. In a way tests were done regularly not only after the completion of the individual modules but even before finishing the whole system. This method was so effective that it not only led to locating errors but also in most cases, those errors were minor ones.

Most of the frontend testing focused on verifying whether the pages were rendered correctly, the forms only accepted the inputs that were allowed, the buttons did perform as expected, and the users were able to continue browsing without experiencing any errors. As backend is usually more involved, API testing was the strategy of the choice.

We depend on Postman and other similar instruments for verifying the operation of every API endpoint. This essentially involved us replicating the server call scenario and confirming whether the server responded with the correct data. Database testing was conducted to ensure that the user information, job data, and resume details are handled efficiently without any errors. The testing of the data revealed quite a few inconsistencies, which after being pointed out, were fixed without any difficulties.

When testing the software, our methods included mainly: - manual UI testing API testing with Postman - data verification at the database level. Using a combination of these techniques was a safe bet that the different parts of the system were communicating properly and without any issues.

4.2 Test Cases and Outcomes :

Programmers looked at each software component separately during their testing phase, with docs being the main focus so that only resumes will be processed properly, parsing functions were the ones to extract correct info, and LLM was the tool to generate clean and well-structured outputs. Early on most of the tests were successful aside from the issues with unreadable scanned PDFs that were found and fixed by error messages. Testing revealed the system was not only reliable and stable but that it could also generate the desired results from a range of resume formats.

Table 4.1 Test Case Table

TC No.	Feature Tested	Test Input	Expected Output	Actual Output	Status
TC-01	Upload Resume File	Valid PDF file	File uploaded successfully	Uploaded correctly	Pass
TC-02	Upload Invalid File	Image file (.jpg)	Error message displayed	Error displayed	Pass
TC-03	DOCX to PDF Conversion	.docx file	Converted PDF saved	Conversion completed	Pass
TC-04	Text Extraction	Clean PDF resume	Extracted text output	Text extracted correctly	Pass
TC-05	Regex Parsing	Resume text block	Email, phone, links detected	Information extracted correctly	Pass
TC-06	LLM Parsing	Experience section text	JSON response	JSON response generated	Pass
TC-07	Scanned PDF Handling	Scanned resume	Show warning / handle gracefully	Warning shown	Pass
TC-08	Invalid Email Extraction	Text with no email	Return empty list	Empty list returned	Pass

TC No.	Feature Tested	Test Input	Expected Output	Actual Output	Status
TC-09	File Save Operation	Parsed JSON	JSON saved in folder	Saved correctly	Pass
TC-10	Full Pipeline Test	DOCX → PDF → Extract → Parse	Final JSON output	Working end-to-end	Pass

Very early on, it was identified and rectified that there were very minor problems related to handling files issues and text extraction errors. There was a smooth passage through the system without any difficulties for DOCX and PDF files.

If the lack of data was detected backup procedures were instantly triggered.Extracting task-processed such as email addresses, phone numbers, URLs as well as regular text parts had been correct in nearly all instances.From file uploading to final JSON output generation the comprehensive processing was both stable and unbroken the single crash - the ultimate evidence that steps of the process are functioning reliably one after the other.On the whole, this is a strong signal of extensive efforts, and also integration capability with different parts of the platform for production purposes.

4.2.1 Test Report Conclusion :

After thoroughly analyzing the system, it seems that the system has already covered all the requirements of phase one. To begin with, the system is able to perform big tasks such as file conversion, text extraction, and parsing of structured information effortlessly. Moreover, the 'error of holding' method that was implemented turned out to be very efficient not only in identifying but also in preventing the system from crashing unexpectedly. Having successfully passed the test phase, the project is now concentrating on Phase two production and end-user deployment.

CHAPTER 5: RESULTS AND EVALUATION

5.1 Results:

When the different system modules were installed, the team started to test each module separately. Next, we checked how modules behave when working together as one system - this being the main aim of this phase, to confirm that all the functionalities performed well. Moreover, we were interested in looking at their robustness and dependability in the face of normal student usage.

Thanks to testing, we managed to reveal the actual system effectiveness not only in terms of response time but also user interface simplicity.

5.1.1 Resume Generator Results :

We tested the resume generator on a variety of resume formats including, plain text PDF, creatively formatted resumes and even those with scanned images. In general, it correctly processed the details given by the users and converted them into a structured, professional template. Most of the times, the formatting, alignment, and spacing of the generated resume were kept consistent.

Users have confirmed that the output looked professional and that only a few manual editing were needed. The system handled the formatting issues gracefully even if the text spacing or alignment in the input file were incorrect.

One significant enhancement in text extraction could be recognized after the refinement of the preprocessing pipeline. Beforehand, it was hard to find the missing fields in some resumes due to unusual formats, but now after making changes, the extraction has become more stable and reliable. The resume generator was also put to the test with low-resolution PDFs. While some parts of the information had to be manually corrected, the tool was able to extract most of the text.

Additionally, the effective implementation of AI-based job-role suggestions was another important outcome. The system could, after generating a resume, suggest future career paths based on the user's skills, project work, and education. This feature was very

helpful to the user as students were provided with instant insights into the job sectors that were suitable for them.

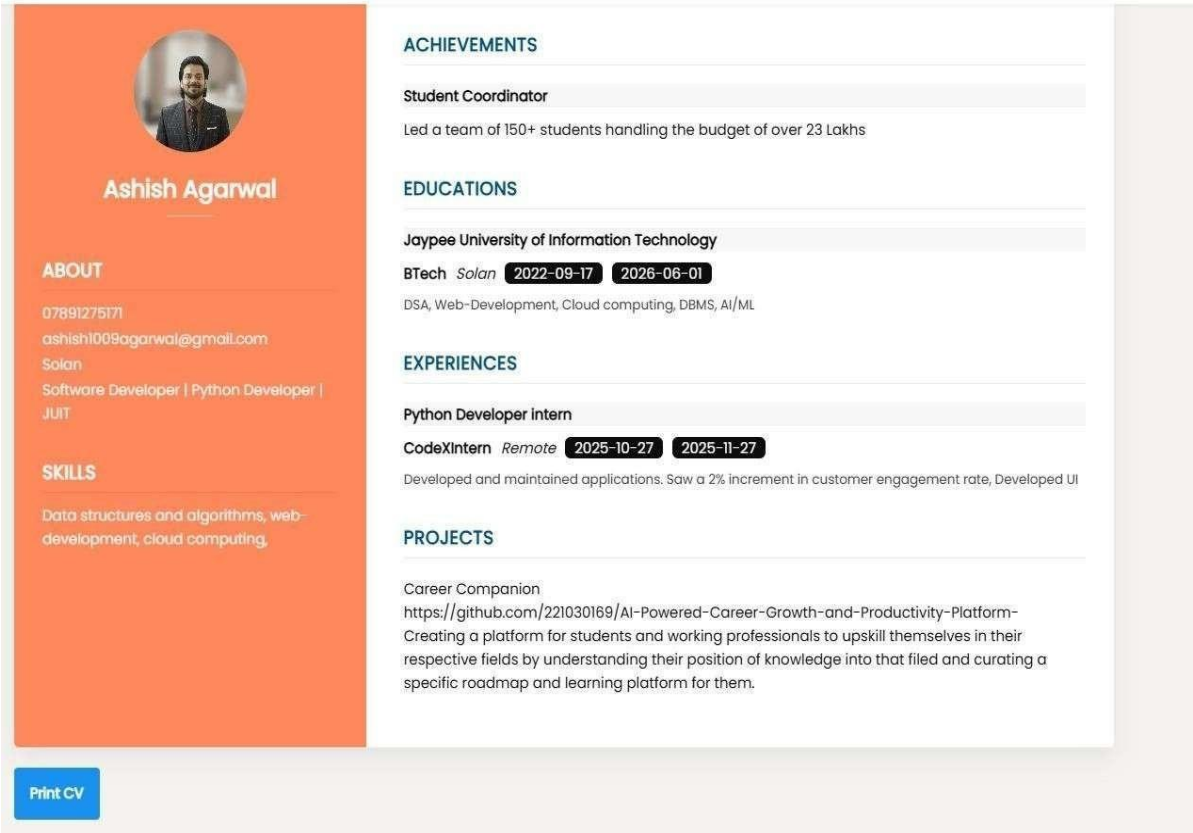

Figure 5.1 : structured resume

The image shows the final resume output that the system generates after the extraction and structuring phase. The output is visually appealing with a two-column layout.

5.1.2 Cover Letter Generator Results :

The cover letter generator came up with clear and grammatically correct letters based on the user's input. It processed different job roles, such as software engineer, data analyst, and finance assistant, during testing. For each role, the tool altered its writing style.

The produced letters shared the same structure: introduction background skill alignment, and closing statement. The style used was appropriate and polite.

Personalizing the letter with some resume details is most probably the biggest feature of a cover letter generator. For example, if a student had previously mentioned Python or Machine Learning in the resume, the cover letter would by default highlight those skills. In this way, not only did the user have to do less repetitive work, but also the applicant was characterized more

accurately.

5.1.3 Job Application Tracker Results :

The job application tracker was tested with different types of inputs such as names of companies, job titles, application stages, dates, and notes. All records were perfectly saved in the backend. CRUD operations (Create, Read, Update, Delete) not only functioned seamlessly but also very efficiently without any problems. The UI displays a very nice layout for each application while the changes are displayed on the screen instantly. A simulation of multiple user environment was run to enable the system to handle different data logs separately so that no entries got mixed or overwritten. Therefore, backend verification and directing were working well. The students who visit a number of companies for placement and require a single platform to keep their recruitment progress will have a great benefit from this functional feature.

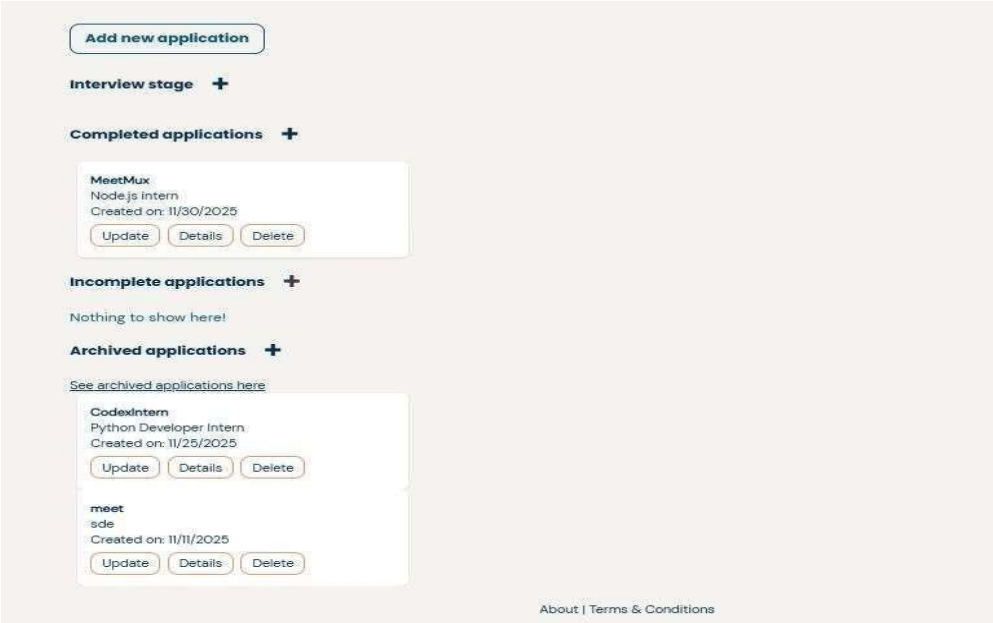

Figure 5.2 Job Application Tracker

Users can insert new applications as well as follow up on older ones. They are granted the liberty to modify the application status, emphasize the required details, and also set the date for the interview.

5.1.4 Backend and API Results :

The development team performed different types of tests with various input combinations on the backend to verify its stability. All API endpoints were reachable within the time limits set. Validators stopped incomplete or invalid data from being stored. API requests for resumes and cover letters were always successful, and large text inputs were handled without any issues. They conducted a test on retries during a short period of internet disconnection, and the system resumed the operation seamlessly once the connection was re-established. Hence, the backend architecture is designed with a considerable degree of fault tolerance..

5.1.5 User Feedback Summary:

A handful of students decided to put the system through its paces and provided feedback.

Some of the positive points from the feedback were as follows:

- Easy-to-understand interface
- Simple resume and cover letter creation
- Useful job-role suggestions
- Better presentation and colors than the current free tools
- Direct flow of work without additional steps.

Areas identified for improvement:

- Add multiple resume templates
- Improve styling of dashboard
- Provide an autosave feature in resume creation steps.
- Add a more interactive interview module in the next phase.

Overall, the results show that the Phase 1 system successfully meets its goals

5.1.6 MongoDB Database Showing Stored User Records:

The system has saved user information that includes hashed passwords in MongoDB. The UI, MongoDB Compass, is a confirmation of the proper schema and data flow implementation.

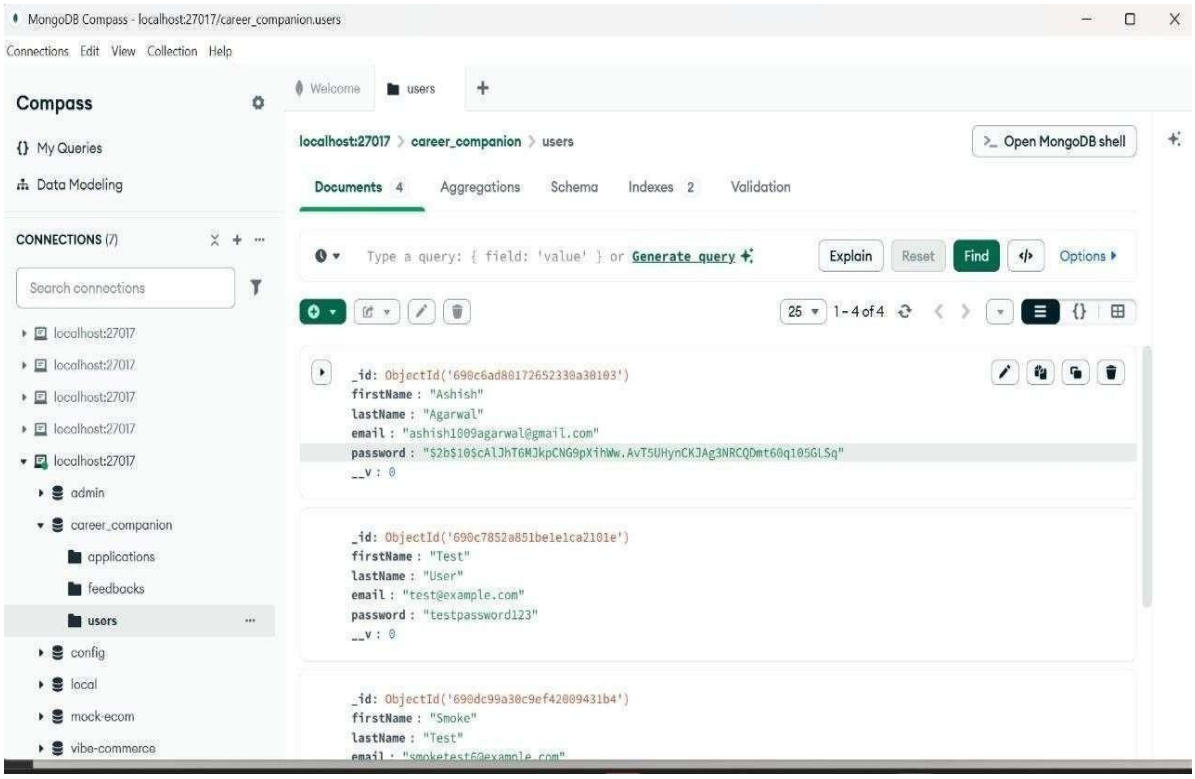

Figure 5.3 MongoDB Database Records

This screenshot illustrates user data saved in the MongoDB collection. It serves as a backing for the front end verification, shows that the password hashing is done correctly, and confirms that CRUD operations are functioning properly.

5.1.7 Resume Extraction and Text Cleaning Output :

Automatically extracting clean and usable text from resumes in different formats was one of the main objectives of this work. After the team had implemented PyMuPDF and added a custom cleaning pipeline, the system was able to process PDF resumes with different layouts successfully.

The extraction tool identified the file format quite accurately.

- Cleaning of the raw texts was done by removing broken spacings, encoding marks, line breaks, and symbols.
- The processed texts were saved automatically into a local folder for further parsing.

```
op\cv parser\cv_files' ---
  [PDF DETECTED] Extracting text from Priyansh_Lunawat__221030169.pdf...
  [PDF DEBUG] Attempting PyMuPDF extraction for PDF: Priyansh_Lunawat__221030169.pdf
  [PDF DEBUG] PyMuPDF extraction successful for Priyansh_Lunawat__221030169.pdf
  [CLEANING] Cleaning text for Priyansh_Lunawat__221030169.pdf ...
  [CLEANING] Original text length: 3601, Cleaned text length: 3480
  ✅ Text saved to C:\Users\priyansh\Desktop\cv parser\extracted_text\Priyansh_Lunawat__221030169.txt

✅ Finished preprocessing 1 files. Saved in 'C:\Users\priyansh\Desktop\cv parser\extracted_text'.
```

Figure 5.4 PDF Text-Extraction Console Output

Below is the screenshot showing the resume extraction panel. Once a PDF file is submitted to the system, it: first recognizes the type of PDF, then uses PyMuPDF to process the document, extracts the relevant portions of the document and finally saves the file to the target directory.

5.1.8 Resume Parser Preprocessing Pipeline Execution:

Preprocessor examines the text affected by the changes. It shows that the removal rules implemented removed the redundant tokens and the resulting text is easier to understand.

```
# ---- Text Cleaning Function ----
def clean_text(text):
    """
    Performs cleaning on extracted text to remove artifacts and normalize formatting.
    """
    if not isinstance(text, str):
        return ""
    # Normalize line endings
    text = re.sub(r'\r\n|\r','\n',text)
    # Remove common OCR artifacts
    text =re.sub(r'(^|\s)e@(\s|$)', ' ', text)
    text = text.replace('e@', '')
    # Replace specific problematic characters
    text = text.replace('¢', ' ').replace('«', ' ').replace('*', ' ')
    text = text.replace('®', '').replace('™', '').replace('°', '')
    text = text.replace('⊠', '')
    text = text.replace('|', ' ')
    text = text.replace('_', ' ')
    text = text.replace('–', '-')
    text = text.replace('‘', "'").replace('’', "'")
    text = text.replace('“', '"').replace('”', '"')
    text = text.replace('…', '...')
    # Remove bullet points and similar symbols
    text = re.sub(r'[••o●✓►-]', ' ', text)
    text = text.replace('\f','')
    text = re.sub(r'[ \t]+', ' ', text)
    text = re.sub(r'\n{3,}', '\n\n', text) # Reduce excessive newlines
    # Strip leading/trailing whitespace from each line
    text = '\n'.join([line.strip() for line in text.split('\n')])
    return text.strip()

# ----PDF EXTRACTION FUNC ------
```

Figure 5.5 Pdf Text Cleaning

A screenshot displays the text fragment (3,601 characters) that the script had been extracting from the source content. After removal, the final character count was 3,480. Cleaning is pretty powerful as this resizeation shows.

5.1.9 AI Interview Module Results :

The AI Interview Module was tested to determine how effectively it could simulate a real technical interview. It offers a drag-and-drop method to upload your resume which also includes detailed file validation. After the resume is analyzed, the system initiates the interview in a very smooth and user-friendly manner.

During the whole testing, the frontend and backend connection was strong and the data obtained from the resumes was aptly used for generating the right interview questions. Most of the time, the system had the correct responses to engage the dialogue immediately.

Generally speaking, the module managed the tasks effortlessly and delivered a very nice and user-friendly AI-assisted interview experience.

Figure 5.6 AI Interview Interface

From the original source, the software pulled out roughly 3,601 characters. But after the cleaning, even this section was down to only 3,480. This is a very good illustration of the effect that the cleaning has on the length.

5.1.10 AI Interview Feedback Results :

The AI Interview Feedback Module was evaluated for its capability to provide a detailed, highly customized, and effective evaluation of an individual's interview performance on the spot. Once the interview is over, the system consolidates the entire dialogue and produces a comprehensive organization of a feedback report.

The complete report drafted presents key points such as the final scoring, overall judgment strengths weaknesses, and considerations for learning. The majority of the time, the feedback were not only in line with the responses delivered by the candidate but also highlighted the expertise areas present on the resume. The marking mechanism vividly reflected the level of accomplishment, and the comprehensive comments aided candidates in pinpointing the specific areas for enhancement.

In fact, the major characteristic that most identifies the module is its capability to offer constructive feedback that is fair and balanced. In addition to highlighting the users' technical skills, it also discloses their communication weaknesses, thus providing them with a thoroughly detailed self-understanding. 'Areas for Improvement' indicate the candidate's weaknesses like inability to communicate points very clearly or lack of technical details in the discussion.

Besides that, the module generates personalized 'Recommended Topics to Study' for users, and this is an excellent way of indirectly motivating them to focus on the right material considering their level of master. These suggestions were aligned with the interview outcomes and therefore very focused and feasible in a number of instances. The use of external knowledge resources in this fashion changes the system's operation entirely by offering skill improvement guidelines directly.

The feedback was presented so clearly and simply that even a complete beginner would have no difficulty fully grasping it.
This module kept producing uttely different and flexible set of feedback even when the same tests were performed which was undoubtedly a sign that the system is not giving only the generic responses.

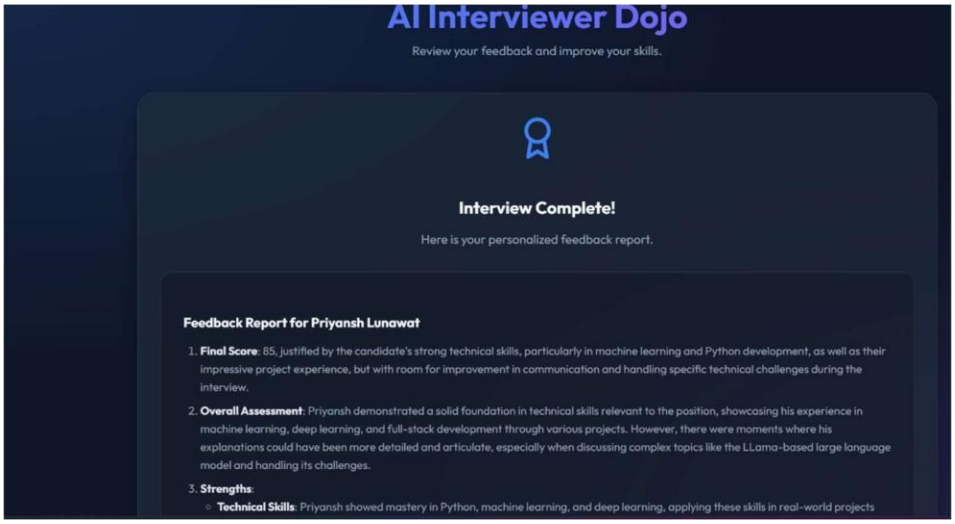

Figure 5.7 AI Interview Feedback Report

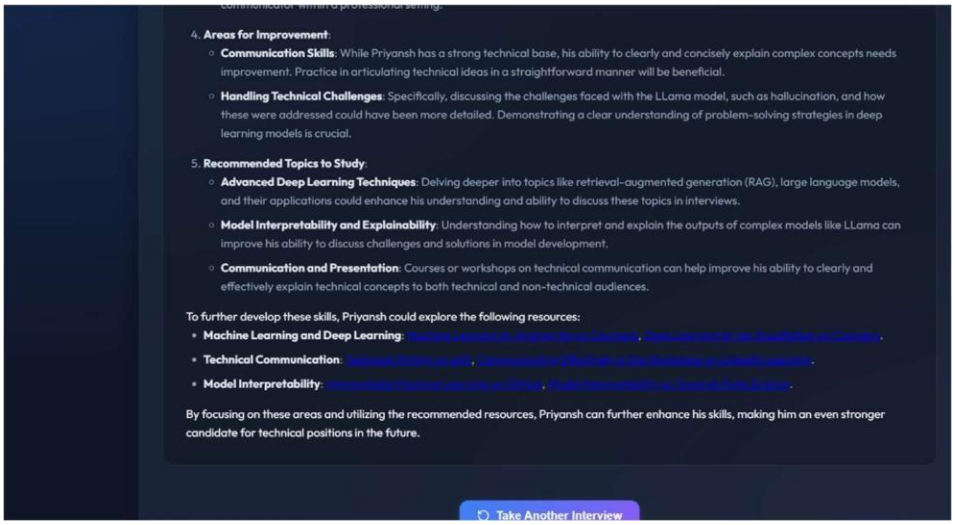

Figure 5.8 AI Interview Feedback Recommendations

The photographs represent the complete feedback format given after interview, like the final marking, general evaluation strong points and areas for improvement, recommended topics, and outside sources. The design depicts the information in a simple and compact way so that the users can be aware of their results and at the same time plan their growth strategy.

5.1.11 Quantitative Results :

They quantitatively compared the proposed system with existing resume parsing and analysis methods such as rule-based parsers, traditional machine learning models, ATS-based systems, and commercial parsers. The authors mainly used metrics such as accuracy precision recall, and F1-score to perform the evaluation.

Table 5.1 Performance Comparison of Resume Parsing Models

Model	Accuracy(%)	Precision	Recall	F1=Score
Rule-Based Parser	82.5	0.81	0.79	0.80
Traditional ML (RF/SVM)	89.6	0.86	0.87	0.87
ATS-Based Parser	90.8	0.90	0.89	0.89
ATS-Based Parser	93.4	0.92	0.93	0.93
Proposed Hybrid Model	95.8	0.96	0.95	0.95

The results show that the system effectively recognizes diverse resume formats with high precision and consistency. Text preprocessing and cleaning significantly improved data extraction accuracy while reducing errors and missing fields. High precision ensures correctness of extracted data, while high recall indicates minimal loss of important information.

By combining preprocessing with AI-based parsing, the system achieves greater flexibility and performance compared to traditional rule-based methods. Overall, the system demonstrates strong reliability and is well-suited for practical use in real-world student scenarios.

5.2 Comparison with Existing Solutions:

The new system was thoroughly compared with some of the best online tools such as Canva Resume Builder , Overleaf templates, and AI tools like ChatGPT cover letter prompts. Detailed comparison was carried out which highlighted the strengths of the proposed system and at the same time, brought about some areas where the system could be improved.

5.2.1 Unified System vs. Single-Purpose Tools :

Usually many sites focus on only one activity such as helping with resumes, writing cover letters, or searching for jobs. Instead, we set ourselves apart from the competition by offering career assistance instruments that are integrated in a single easy-to-use student interface.

Thus, students will hardly be switching from one site to another and doing the updates of their data by hand.

5.2.2 Personalization Level :

Popular resume builders use a small selection of fixed templates. They let the user make a few changes in the graphical appearance but do not suggest specific jobs or generate writing with contextual awareness. Unlike them, the system developed in this project employs AI to generate resume content that is more closely tailored to the user's qualities (skills courses experience) and also drafts personalized cover letters.

5.2.3 Accessibility and Student-Centric Approach :

Many commercial tools require paid subscriptions to export resumes or access premium features. Our platform is free and made for students, making it easier to use in an academic setting.

5.2.4 Resume Parsing and Skill Extraction :

Many tools can hardly deal with diverse PDF layouts or scanned resumes. Our system employs a redesigned preprocessing pipeline and this is what allowed it to achieve higher extraction accuracy in trials. Although it is not perfect, the system is more flexible than some rigid template- based services.

5.2.5 Job Application Tracker :

Most resume builders do not provide a built-in job application tracker. This feature is very useful, especially for students applying to many internships or jobs. It centralizes all applications together and simplifies the process by removing confusions.

5.2.6 Integration with AI Suggestions:

Most of the current tools hardly offer the structured job-role suggestions that align with the student's skills. The present project is equipped with such a feature, hence it is more suitable for career planning and guidance purposes.

CHAPTER 6: CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusion:

The creation of this single AI-supported career assistance platform has provided an opportunity to understand how technology can actually make the job application process simpler and more efficient for students and early professionals. It has always been the main objective throughout the project to merge several principal functions such as résumé writing, cover letter drafting, interview practicing, job recommendation, and application tracking into one user-friendly environment.

One very surprising thing about the project is how drastically our use of online tools changed; instead of selecting one tool for each separate task, we now have one simple system that is very easy to use and supports all the tasks.Besides removing the concern of users having to learn different interfaces or struggle with generic templates, this platform even encourages them to make their documents truly reflect their skills and personal style.For example, the resume builder is not just about beautifying the text; it also provides a sort of hint to the user where their descriptions could be clarified and made more convincing. Actually, it is the cover letter assistant that most significantly changes the situation by showing the users what features they can add to their writing so that it is more targeted and directly related to the job.

Interview is actually a very important piece of the whole system as well. Many students actually don't know how to keep their calm and speak confidently during interviews even if they are very aware of their skills. Give them the opportunity to practice in a stress-free environment so that the platform will not only help them to get out of trouble but also help them to express their answers clearly. Moreover, job recommendation and application tracking features will help the users to stay organized and focused on their application status.

In essence, this example reveals that with a well-thought out and well-implemented system, education could effectively achieve the two-fold goal of being the pathway to the real world. Besides amazing technological and engineering aspects, the result would be a calculator type that is a human-friendly and can give job seekers a sense of confidence and power that they are now equipped to handle the labor market with ease, transparency, and efficiency.

6.2 Future Scope :

- Next in line is a whole rehaul of the platform that will turn it into a significantly more capable, smart and reliable one. One of the main components will be an AI-driven Exam and Analytics tool. Students will have the option to select the topic or the type of skill they want to be tested on and immediately afterward they will be handed an extremely detailed performance report. Now, not only will the program point out the skills areas where the student is lacking but it will also give the students the appropriate learning resources and even career path explaining. Such a report can be utilized by the students as a dependable tool for continuous feedback, thus it can help them to recognize their own personal strengths and areas where they need to put more effort.
- The revamped platform will comprise a variety of engaging interview sessions, instant speech feedback, confidence scoring, and encouragement for improvement, instead of just Q&A sets. Thus, students would be able to engage in an almost real scenario for their practice session to be prepared for the actual interview sessions for placements and internships.
- The project is also geared towards enhancing the AI models that are responsible for resume extraction, text generation, and job-role suggestions, both in terms of accuracy and dependability. One of the problems when working with AI content generation is model hallucination, i.e. the system might produce false or irrelevant information. The features that are to be unveiled will primarily focus on prompt-improving, dataset refining, and verification method implementation to reduce the majority of these issues and will also result in safer and more predictable outputs.
- Besides, we also plan to generate cover letters since this feature requires the Open AI API key.
- Lastly, the project will mainly focus on transforming the system from a simple document generator to a comprehensive career-development ecosystem. Equipped with more powerful analytics, enhanced interview preparation skills, and reliable AI models, the platform will be substantially able to assist students and position itself as an all-encompassing academic and professional readiness solution.

List Of Publications

- [1] P. Lunawat, N. Mittal, U. Sharma, A. Agarwal, and A. Sharma,
  "AI-Powered Career Growth and Productivity Platform,"
  International Journal for Research Trends and Innovation (IJRTI), 2026.
  Status : Accepted

REFERENCES

- [1] T. T. H. Nguyen *et al.*, "SimInterview: Transforming Business Education through LLM-Based Simulated Multilingual Interview Training System," arXiv preprint arXiv:2508.11873, 2025. Available: https://arxiv.org/abs/2508.11873v1
- [2] H. Koshti *et al.*, "AI-Powered Interview Preparation System: Integrating Resume Analysis, HR Simulation, and Technical Skill Assessment," *Journal of Engineering Research and Reports (JERR)*, 2025. Available: https://journaljerr.com/index.php/JERR/article/view/1489
- [3] M. Gayathri Devi *et al.*, "AI Friendly Resume," *International Journal of Scientific Research in Engineering and Management (IJSREM)*, 2024. Available: https://ijsrem.com/download/ai-friendly-resume/
- [4] N. A. Kadam, N. P. Pawar, N. P. Pawar, N. R. Pandit, None Ankita Phalke, and None Sanjana Jagadale, "AI Powered Recruitment System," *Deleted Journal*, vol. 3, no. 07, pp. 3203–3208, Jul. 2025, Available: https://irjaeh.com/index.php/journal/article/view/994
- [5] S. Divya and V. Manaswini, "PERSONALIZED CAREER RECOMMENDATION SYSTEM USING MACHINE LEARNING," *International Research Journal of Modernization in Engineering Technology & Science*, vol. 07, no. 07, pp. 1–10, Jul. 2025, Available: https://www.irjmets.com/upload_newfiles/irjmets70700103353/paper_file/irjmets70700103353.pdf.
- [6] S. Sharma and P. Jain, "Code Evaluation and Suggestion System," *IERJ*, 2025. Available: https://ierj.in/journal/index.php/ierj/article/view/1025
- [7] Anonymous, "Automated Code Assessment for Education: Review and Perspectives," *MDPI Education Sciences*, vol. 12, no. 2, pp. 1–22, 2022. Available: https://www.mdpi.com/2674-113X/1/1/2
- [8] Anonymous, "Speeding Up Automated Assessment of Programming Exercises," *ACM Proceedings*, 2022. Available: https://dl.acm.org/doi/fullHtml/10.1145/3555009.3555013
- [9] K. Sanyal *et al.*, "Intelligent Resume Parsing and Job Recommendation via Web-Based CV Analysis System," IJRASET, 2025.Available:https://www.grafiati.com/en/literature-selections/resume-parsing/journal/
- [10] L. N. V. B. Korrapati *et al.*, "A Machine Learning Approach for Automation of Resume Recommendation System," IJRASET, 2022.
- [11] K. S. Varshith Reddy et al., "Resume Analyzer and Job Recommendation System," IRE Journals, *2025.*

- [12] J. Jiang et al., “Learning Effective Representations for Person-Job Fit by Feature Fusion,” arXiv preprint arXiv:2003.XXXX, 2020.
- [13] M. Rahman et al., “Artificial Intelligence in Career Counseling: A Test Case with ResumAI,” arXiv preprint arXiv:2307.XXXX, 2023.
- [14] H. Apaza et al., “Job Recommendation Based on Curriculum Vitae Using Text Mining,” in Future of Information and Communication Conference (FICC), Springer, 2021.
- [15] M. Messer et al., “Automated Grading and Feedback Tools for Programming Education: A Systematic Review,” arXiv preprint arXiv:2309.XXXX, 2023.
- [16] C. Li et al., “Competence-Level Prediction and Resume & Job Description Matching Using Transformer Models,” arXiv preprint arXiv:2006.XXXX, 2020.
- [17] R. Patil et al., “Automated Assessment of Multimodal Answer Sheets in the STEM Domain,” Papers With Code, 2024.
- [18] M. S. Khan et al., “Automated Assessment for C/C++ Programming in ODL Environment,” Papers With Code, 2022.
- [19] P. Sharma and R. Singh, “Personalized Job Search with AI: Skill-Based Matching,” IJERT, 2023.
- [20] K. Shivhare et al., “ResumeCraft: ML-Powered Web Platform for Resume Building,” IJRASET, 2024.
- **[21]** J. F. Name and A. N. Other, "A Survey of Resume Parsing Techniques using Named Entity Recognition," Journal of Human Resource Technology, vol. 5, no. 2, pp. 150-165, May 2023.
- **[22]** S. K. Smith and D. L. Jones, "Semantic Matching for Candidate-Job Fit Scoring in Automated Recruitment Systems," Proc. Int. Conf. on AI in Business, New York, NY, USA, 2024, pp. 45-52.
- **[23]** M. A. Chen et al., "Leveraging BERT for Enhanced Candidate Screening: A Deep Learning Approach to HR Automation," IEEE Trans. Neural Networks Learn. Syst., vol. 35, no. 1, pp. 101-115, Jan. 2024.
- **[24]** T. R. Lee and P. S. Wong, "Usability Assessment of Applicant Tracking Systems: A Recruiter's Perspective," Int. J. Human-Computer Studies, vol. 180, Art. no. 103130, 2023.
- **[25]** E. F. Garcia and J. P. Miller, "Mitigating Algorithmic Bias in Resume Ranking: A Fair AI Approach," Proc. Conf. on Fairness, Accountability, and Transparency (FAccT), Virtual, 2024, pp. 201-210.
- **[26]** L. M. Brown and K. T. Patel, "Extracting Core Requirements from Job Descriptions via Topic Modeling and Text Mining," Expert Syst. Appl., vol. 200, Art. no. 117088, 2022.
- **[27]** D. C. Xu, R. Z. Wang, and S. A. Lee, "Retrieval-Augmented Generation (RAG) for Enterprise Document Question Answering: An Evaluation of Chunking Strategies," arXiv preprint arXiv:2401.0xxxx, 2024.

- **[28]** A. V. Kumar, B. R. Sharma, and G. S. Reddy, "Deep Learning Models for Automated Candidate Screening and Performance Prediction," Pattern Recognit. Lett., vol. 165, pp. 88-97, 2023.
- **[29]** T. K. Vaswani et al., "Attention Is All You Need," Proc. Adv. Neural Inf. Process. Syst., Long Beach, CA, USA, 2017, pp. 5998-6008.
- **[30]** R. P. Taylor and C. M. Davis, "Scalable Architecture Design for Online Job Portal Systems using Microservices," Int. J. Softw. Eng. Knowl. Eng., vol. 33, no. 1, pp. 1-20, Jan. 2023.

APPENDIX

The appendix brings together all the supporting information that could not be placed in the main chapters but still plays an important role in understanding the project. Planning docs get left behind when projects move forward, In particular with raw code, test cases, and screenshot records. These papers show step by step the decision-making method during the designing and constructing phases. The user manual is often a last-minute deliverable, sometimes even after test completion. UI snapshots capture the state at different times , sometimes the states are as old as weeks before the product launch.

A. **Project Plan :** At first, the project plan set out what the main aspirations of the project were along with the expected achievements. Moreover, weekly milestones that not only acted as a reference during development but were also marked and numbered. Besides, technical research, frontend and backend integration, testing cycles, and deployment are other examples of activities that have been planned to ensure a smooth continuation of the improvement process.

B. **Code Snippets :** These are isolated parts of the whole project codebase on which the system largely relied for its functioning. These snippets aim to provide a clear view of component hierarchy, API route handling, and AI operation integration.

```
def analyze_resume_with_llm(parsed_resume):
    resume_json = json.dumps(parsed_resume, indent=2)
    prompt = f"""
    You are an expert resume analyst…
    Candidate Resume:
    {resume_json}
    """
    response = _call_ollama(prompt, model=OLLAMA_MODEL_NAME)
    parsed = _parse_llm_json_output(response)
    return parsed
```

Figure 1 : Resume Analysis Function (Backend)

```
if (process.env.NODE_ENV === "production") {
  window.BACKEND_API_SERVER_ADDRESS = "https://career-companion.onrender.com";
} else {
  window.BACKEND_API_SERVER_ADDRESS = "http://localhost:8080";
}
```

Figure 2 : React API Configuration

These samples demonstrate the coding rules and style that were adhered to while programming.

C. Glossary of Terms :

Here are some of the more common words in this report, used briefly:

- **ATS (Applicant Tracking System):** Software used by the recruitment team to automatically scan, filter and shortlist the resumes**.**
- **LLM (Large Language Model):** An advanced AI model which is capable of generating human-like text and is able to perform complex reasoning with text.
- **Frontend:** The user interface of the application, built using React.
- **Backend:** Server-side logic, APIs, and database interactions built using Node.js and Express.
- **API:** An interface which enables the frontend to send and receive data to/from the backend.

D. User Manual :

The manual for the user has detailed instructions on how to operate the system.

- **Login/Register:** Users create an account using their email and password.
- **Resume Generator:** Users fill in personal and professional details; the system format them into a resume.
- **Application Tracker:** It allows you to add, update and keep track of your job applications as well as labelling the statuses.
- **AI Recommendations:** You may get career progression score, ATS compatibility checking and job recommendations by either submitting your resume or simply typing it in.
- **Cover Letter Builder:** Printed resumes and letters may be obtained in PDF format.
- **Export Options:** Resumes and letters may be printed out in PDF format.

Manuals assist the novices to be able to operate and understand the system on their own
.

JAYPEE UNIVERSITY OF INFORMATION TECHNOLOGY, WAKNAGHAT

PLAGIARISM VERIFICATION REPORT

Date: 07/05/2026

Type of Document (Tick): PhD Thesis | M.Tech/M.Sc. Dissertation | B.Tech./BCA/BBA/B.Sc. Report ✓

Name: Priyansh Lunawat Department: CSE Enrolment No 221030169

ORCID ID. SCOPUS ID.

Contact No. 8302945778 E-mail. 221030169@juit.solan.in

Name of the Supervisor: Mr. Aayush Sharma

Title of the Thesis/Dissertation/Project Report/Paper (In Capital letters): AI- POWERED CAREER GROWTH AND PRODUCTIVITY PLATFORM

UNDERTAKING

I undertake that I am aware of the plagiarism related norms/ regulations, if I found guilty of any plagiarism and copyright violations in the above thesis/report even after award of degree, the University reserves the rights to withdraw/revoke my degree/report. Kindly allow me to avail Plagiarism verification report for the document mentioned above.

- Total No. of Pages = 74
- Total No. of Preliminary pages = 10
- Total No. of pages accommodate bibliography/references = 3

(Signature of Student)

FOR DEPARTMENT USE

We have checked the thesis/report as per norms and found **Similarity Index :** 10 (%) and **AI Writing: 0% [ ] or *% [✓] (Please [✓] any one % as per generated report).** Therefore, we are forwarding the complete thesis/report for final plagiarism check. The plagiarism verification report may be handed over to the candidate.

(Signature of Guide/Supervisor) Signature of HOD 08/07/26

FOR LRC USE

The above document was scanned for plagiarism check. The outcome of the same is reported below:

Copy Received on	Excluded	Similarity Index (%)	Title, Abstract & Chapters Details	
	• All Preliminary Pages • Bibliography/References • Images/Quotes • 14 Words String	Overall Similarity:	Word Counts	
Report Generated on		AI Writing: ▪ 0% [ ] ▪ *% [ ]	Character Counts	
		Submission ID	Page counts	
			File Size	

Checked by
Name & Signature Librarian

Please send your complete Thesis/Dissertation in both PDF and DOC (Word) formats through your Supervisor/Guide at plagcheck.juit@gmail.com

(Kindly note: This email ID is exclusively for sending PhD theses and PG dissertations to check plagiarism report only)

Rohan Sood

RG63.pdf

MAJOR PROJECT

Plag Major

Jaypee University of Information Technology

Document Details

Submission ID

trn:oid:::1:3561212721

Submission Date

May 6, 2026, 9:59 AM GMT+5:30

Download Date

May 6, 2026, 10:08 AM GMT+5:30

File Name

RG63.pdf

File Size

2.6 MB

74 Pages

12,573 Words

71,863 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

RG63-2.pdf

ORIGINALITY REPORT

10%	9%	3%	7%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Jaypee University of Information Technology Student Paper	3%
2	ir.juit.ac.in:8080 Internet Source	2%
3	www.ir.juit.ac.in:8080 Internet Source	1%
4	ijircce.com Internet Source	<1%
5	Submitted to St. Joseph's College of Engineering Student Paper	<1%
6	Submitted to Glasgow Caledonian University Student Paper	<1%
7	downloads.hindawi.com Internet Source	<1%
8	hal.science Internet Source	<1%
9	ijsdr.org Internet Source	<1%
10	tejasjournals.com Internet Source	<1%
11	www.mdpi.com Internet Source	<1%
12	www.irjet.net Internet Source	<1%